SMART HELMET FOR INDUSTRIAL WORKER SAFETY USING EMBEDDED SENSORS

POH ZHENG YANG

ANSON KOK MING JING

LEE JOON YI

NG YU MENG

UNIVERSITI TUNKU ABDUL RAHMAN

SMART HELMET FOR INDUSTRIAL WORKER SAFETY USING EMBEDDED SENSORS

POH ZHENG YANG
ANSON KOK MING JING
LEE JOON YI
NG YU MENG

A project report submitted in partial fulfilment of the requirements for the award of Bachelor of Engineering (Honours) Electronics Engineering

Faculty of Engineering and Green Technology
Universiti Tunku Abdul Rahman

February 2026

APPROVAL FOR SUBMISSION

I certify that this project report entitled **"SMART HELMET FOR INDUSTRIAL WORKER SAFETY USING EMBEDDED SENSORS"** was prepared by **POH ZHENG YANG, ANSON KOK MING JIN, LEE JOON YI and NG YU MENG** has met the required standard for submission in partial fulfilment of the requirements for the award of Bachelor of Electronics Engineering with Honours at Universiti Tunku Abdul Rahman.

Approved by,

Signature : _____

Supervisor : _____

Date : _____

Signature : _____

Co-Supervisor : _____

Date : _____

The copyright of this report belongs to the author under the terms of the copyright Act 1987 as qualified by Intellectual Property Policy of Universiti Tunku Abdul Rahman. Due acknowledgement shall always be made of the use of any material contained in, or derived from, this report.

© 2026, Poh Zheng Yang, Anson Kok Ming Jin, Lee Joon Yi and Ng Yu Meng. All rights reserved.

ACKNOWLEDGEMENTS

I would like to thank everyone who had contributed to the successful completion of this project. I would like to express my gratitude to my research supervisor, Ts. Dr. Nuraidayani Binti Effendy for her invaluable advice, guidance and his enormous patience throughout the development of the research.

In addition, I would also like to express my gratitude to my loving parents and friends who had helped and given me encouragement.

(This acknowledgements page is optional and shall be treated as reference only)

SMART HELMET FOR INDUSTRIAL WORKER SAFETY USING EMBEDDED SENSORS

ABSTRACT

The Smart Helmet prototype combines environmental sensing, motion indication, BLE proximity and attendance functions with a computer dashboard. Its implementation uses three ESP32 programs: Main for the helmet sensors and warning outputs, Cam for image access, and Receiver for proximity and attendance decisions. The devices and computer communicate over a shared Wi-Fi network. Receiver also sends its selected indication to Main through BLE, allowing Main to report the received state to the computer.

The study examines the implementation supplied in Main.zip and evaluates the available software evidence. Main acquires light, gas, inertial, temperature and humidity, battery and helmet-presence inputs. A common output routine resolves requests in the order gas, impact, temperature and BLE. Receiver estimates distance from signal strength and provides configurable attendance hours, delays and clock controls. The Python application collects telemetry, stores records in SQLite and exports attendance data to CSV and Excel. Camera access and Firebase interfaces are included in the source.

On 16 September 2026, twenty-eight Python test methods passed in 2.070 s using test clients and substituted device responses. The checks support selected host behaviours, including Main-forwarded BLE data handling, duplicate-event rejection, retained manual time and export recovery. This execution did not exercise physical sensors, wireless delivery, camera capture or a live cloud service. The gas outputs remain relative indices, while BLE distance remains an estimate requiring evaluation

in the intended environment. The result is an integrated prototype with documented software verification and a defined plan for physical evaluation. Sensor calibration, warning timing, radio performance and event retention under interruption remain necessary before field-performance claims can be made.

The report also sets out what the prototype costs and what it consumes. An indicative bill of materials places one complete set in the low hundreds of ringgits, which is the figure that decides whether monitoring of this kind is affordable to a small contractor at all. The sustainability discussion identifies the heated gas elements as the real consumable and names the continuous divider drain and the fixed interval uploads as avoidable waste, and the work is then aligned against Goals 3, 8, 9, 10 and 12 of the United Nations Sustainable Development Goals as an alignment of intent and not a measured contribution. Sections reserved for the physical measurements are left deliberately unfilled, with their procedures fixed in advance.

TABLE OF CONTENTS

DECLARATION **i**
APPROVAL FOR SUBMISSION **v**
ACKNOWLEDGEMENTS **vii**
ABSTRACT **viii**
TABLE OF CONTENTS **ix**
LIST OF TABLES **xiii**
LIST OF FIGURES **xv**

CHAPTER

1 INTRODUCTION **1**
1.1 General Introduction 1
1.2 Importance of the Study 1
1.3 Problem Statement and Rationale 3
1.4 Aim and Objectives 3
1.5 Scope and Limitations 5
1.6 Report Organisation 6
2 LITERATURE REVIEW **7**
2.1 Helmet Warning Approaches 7
2.2 RSSI Ranging and Controller Constraints 8
2.3 Sensing and Measurement Validity 10
2.4 Gas Sensing and Calibration Limits 11
2.5 Proximity Attendance and Identity Assurance 12
2.6 Helmet Compliance Detection and Presence Sensing 14
2.7 Local Records, Cloud Storage and Security 15
2.8 Position of the Present Implementation 16
2.9 Environmental Temperature and Heat Stress Interpretation 18
2.10 Impact and Motion Sensing in Helmet Systems 19
2.11 Helmet-Mounted Vision and Remote Assistance 21

3	**METHODOLOGY AND WORK PLAN**		**23**
	3.1	Method and Work Plan	23
	3.2	Requirements and Design Constraints	24
	3.3	System Architecture	26
	3.4	Firmware Task Structure and Timing	28
	3.5	Acquisition and Illumination	30
	3.6	Gas Processing	32
	3.7	Motion Detection	33
	3.8	Temperature and Heat Index	34
	3.9	Battery and Helmet Presence	36
	3.10	Warning Arbitration	37
	3.11	BLE Distance and Operating Modes	39
	3.12	Distance Model and Zone Decision	40
	3.13	Attendance and Receiver Time	42
	3.14	Camera and Local Records	45
	3.15	Camera Subsystem Implementation	47
	3.16	Host Application Architecture	49
	3.17	Cloud Interfaces and Access Controls	51
	3.18	Power Budget and Battery Estimation	52
	3.19	Proposed Physical Evaluation	54
	3.20	As-Built Hardware	55
	3.21	Laptop Attendance, Voice Assistant and Emergency Reporting	58
4	**RESULTS AND DISCUSSIONS**		**60**
	4.1	Execution Conditions	60
	4.2	Telemetry and Record Handling	61
	4.3	Timing and Interface Checks	61
	4.4	Test Summary and Objective Assessment	62
	4.5	Hardware Verification and Outstanding Measurements	65
	4.6	Cost Analysis	73
	4.7	Sustainability Analysis	75
	4.8	Alignment with the Sustainable Development Goals	76
	4.9	Recorded Dashboard Behaviour	78
	4.10	Threats to Validity	81

4.11 Interpretation and Limitations 83

5 CONCLUSIONS AND RECOMMENDATIONS 84

5.1 Conclusions Against Objectives 84

5.2 Recommendations 86

REFERENCES 87

APPENDIX A: INSTALLATION AND OPERATING MANUAL 93

APPENDIX B: SOURCE AND VERIFICATION RECORDS 97

APPENDIX C: FIRMWARE INTERFACES AND CONFIGURATION 100

APPENDIX D: SELECTED SOURCE LISTINGS 103

APPENDIX E: PHYSICAL TEST PROTOCOLS 112

LIST OF TABLES

TABLE	TITLE	PAGE
2.1:	Comparison of Warning Approaches	17
3.1:	Work Plan and Evidence Status	23
3.2:	Project Requirements and Their Evidence Class	25
3.3:	Main Pin Assignments and Program Settings	27
3.4:	Acquisition Intervals and Sample Counts in Main	29
3.5:	Distance Model Parameters and Their Permitted Ranges	41
3.6:	Initial Receiver Configuration	44
3.7:	Principal HTTP Interfaces	44
3.8:	Camera Endpoints and Their Use by the Host	48
3.9:	Host Application Endpoints	50
3.10:	Indicative Power Budget from Published Device Specifications	53
4.1:	Executed Python Checks	63
4.2:	Objective Assessment and Outstanding Evidence	64
4.3:	Bill of Materials at Cytron Listed Prices, 16 September 2026	74
4.4:	Alignment with the United Nations Sustainable Development Goals	78
A.1:	Installation Commands and Configuration	94
A.2:	Operational Troubleshooting	96
B.1:	Source and Verification Artefacts	97

B.2: Evidence Traceability from Report Claim to Source Location 98

C.1: Complete Pin Map of the Main Controller 100

C.2: Receiver Control Interface 101

C.3: Local Database Schema 102

LIST OF FIGURES

FIGURE	TITLE	PAGE
1.1:	Boundary between the work undertaken and the work left outside the project.	5
2.1:	The reviewed work grouped by the question each study answers.	17
3.1:	Work plan as a schedule, with the outstanding calibration and trials hatched.	24
3.2:	Data and control paths as they are implemented across the four programs.	27
3.3:	Sequence of one host acquisition cycle across the four programs.	30
3.4:	Commanded lamp PWM against ambient illuminance, computed from the implemented mapping.	32
3.5:	States and transitions of the two-stage impact detector.	34
3.6:	Computed heat index against air temperature at four relative humidities, with the classification boundaries applied by the firmware.	36
3.7:	Order in which the branches are offered the shared outputs, with one owner per pass.	38
3.8:	Computed distance against filtered RSSI for three path loss exponents, with the default zone boundaries.	42
3.9:	Attendance states held by the receiver and the conditions that move between them.	45

3.10: Interior of the shell, showing the controller board, the converter and the harness as they are being mounted. 56

3.11: Front of the helmet with the two gas elements mounted through the shell and the lamp below them. 57

3.12: Side view of the assembled helmet, where the indicators sit at the top edge of the shell. 57

3.13: Crown of the helmet carrying the BH1750 and the DHT22, with the two indicators above them. 58

4.1: Evidence ledger entries counted by class of evidence. 65

4.2: Red LED lights up during dangerous situations. 67

4.3: Yellow LED turns on when at warning situations. 68

4.4: Both LEDs turn on when at critical situations. 68

4.5: Ambient light shows darker when the surrounding environment is bright. 69

4.6: Ambient light shows brighter when the surrounding environment is dark. 69

4.7: Overview page of the host application, carrying the fourteen indicator cards at the time of the session. 70

4.8: Check-in and check-out panel, with both switches in the off state the host will hold at every start. 71

4.9: Geofencing panel with the helmet inside the red boundary of the beacon. 71

4.10: Camera panel and the stored light history of the same session. 72

4.11: Event register and the connection settings page of the host application. 72

4.12: Cost by subsystem, from the prices listed in Table 4.3. 75

4.13: Overall gas index history for one recorded session. 79

4.14: Geofence zones at increasing distance. Left: red, about 1.8 m. Centre: yellow, about 4.6 m. Right: outside, about 7.1 m. 79

4.15: Full screen warning overlay on the host application. 80

4.16: Overview page with gas, impact, temperature and geofence active together. The alert card reports GAS as the highest priority source. 81

CHAPTER 1

INTRODUCTION

1.1 General Introduction

The Smart Helmet prototype combines local sensing with a computer interface for one helmet. Its design assigns the sensor inputs and warning outputs to Main, uses a separate ESP32-CAM for images, and places BLE scanning and attendance decisions in a receiver. The computer collects the state of these devices over a shared Wi-Fi network. This arrangement allows the receiver to retain its own decision logic when the dashboard is closed, although local records on the computer still depend on the host application running.

1.2 Importance of the Study

Site supervision still depends on a person walking the floor and looking. The current status quo of that arrangement still remains imperfect, since one supervisor cannot watch gas exposure, heat load, a head impact and the attendance register at the same instant, and the written record is being produced afterwards from memory. This project takes those concerns and places them onto the helmet, which is the one item a worker is already obliged to wear for the whole shift.

The importance of the study sits in three places that do not carry equal weight. The first is that the helmet already has a fixed position on the body, hence a sensor mounted there reports the condition at head height where the worker actually breathes, and not the condition at the wall where a fixed monitor usually hangs. Head height matters for carbon monoxide in particular. The second is the record. An attendance sheet signed at a gate proves that somebody wrote a name, when a proximity record written by the receiver proves that a tag was inside a boundary for a measured dwell period, and the two are not the same claim. The third is far weaker and must be stated as such, which is that a prototype of this kind demonstrates feasibility of the integration and nothing about field reliability.

Wearable safety instrumentation is an established direction in construction research, and worker acceptance has been studied directly rather than assumed (Choi, Hwang and Lee, 2017). Proximity alerting between workers and plant equipment was demonstrated as a working system more than a decade ago (Teizer et al., 2010). What has changed since is cost. The controller, the radio and the sensors used here are inexpensive parts that a student can buy, hence allowing the integration question to be asked at a scale that was previously a funded project.

There is also a reason to build the coordination rather than only the sensing. A helmet that beeps at its wearer is useful. A helmet that beeps at its wearer while a receiver holds the authoritative clock and writes an attendance row into a local database, which is then exported to a spreadsheet and pushed to a cloud branch, is a different kind of object, because the evidence survives the shift. Most published helmet work reports the sensing and stops there. The gap being addressed is the one between a reading and a record.

The scale of the underlying problem is worth stating with a source rather than asserted. Fatal occupational injury is counted differently by different national systems, and a comparison of two United States surveillance sources found meaningful differences in how the same events are characterised (Marsh and Jackson, 2013). The relevance here is narrow and is only this, that any claim about how many injuries a

device of this kind might prevent depends first on how injuries are counted, hence this report makes no such claim anywhere.

1.3 Problem Statement and Rationale

The main engineering problem is the interaction between functions. A gas warning and a geofence indication may request the same red LED or buzzer at the same time. Without a common output rule, the visible indication would depend on which module wrote to the pin last. The implemented controller resolves this by assigning gas the highest priority, followed by impact and temperature, before handling BLE indications. The order is part of the software design. Its suitability for a particular workplace requires a separate assessment.

The monitoring interface has a related coordination problem. Sensor values originate from Main, whereas proximity and attendance records originate from Receiver. A computer that reads only the receiver would miss the environmental measurements. Reading only Main would omit the receiver configuration and its full event state. The host consequently polls both devices and stores their records locally. Main also reports the BLE command it received, so the operator can compare the receiver decision with the state reaching the helmet.

1.4 Aim and Objectives

This project aims to implement these functions as a single, inspectable prototype. The objectives cover acquisition of the six sensor modules and their local outputs, camera access, BLE proximity and attendance control, and a dashboard that retains telemetry and exports event records. Verification is considered part of each objective. A function appearing in source code establishes its implementation, but it does not establish the accuracy of a sensor or the reliability of a wireless link.

The objectives are assessed at two levels. At the implementation level, the required paths must be present and connected: a sensor read must reach its decision routine, a receiver event must reach the host record, and a control request must be directed to the correct device. At the execution level, the behaviour must be observed under stated conditions. Keeping these levels separate avoids treating a successful source inspection as a measurement of the assembled helmet.

The first objective concerns local monitoring. Light control, gas indication, motion detection, temperature and humidity, battery estimation and helmet presence are all assigned to Main. Each module has its own sampling or state logic, but the shared warning outputs are resolved together. This makes overlapping conditions an explicit test case. A later physical test must apply combinations of inputs and record which indication remains active, rather than checking each sensor in isolation and assuming the combined behaviour follows.

The second objective concerns proximity and attendance. Receiver must recognise the configured tag, derive a zone, apply the selected operating mode and retain the time settings. The manual clock control is relevant because a demonstration of late arrival or overdue attendance should not require waiting through a full working day. Changing the clock must also have a predictable effect on subsequent state decisions. The software host includes a test for retaining a manually selected time instead of immediately replacing it with computer time.

The third objective is the operator interface. The computer application should show current device connectivity alongside the retained measurements, provide the camera controls, and expose the permitted receiver settings. Local storage is required because a visible reading disappears when the page changes unless the application records it. The event register and spreadsheet exports provide a way to inspect attendance transitions after the live indication has ended. This objective includes handling an export file held open by another application.

1.5 Scope and Limitations

The scope is deliberately one helmet, identified in the BLE application as TAG001. The receiver uses signal strength to estimate proximity, and the attendance controls operate through hours and minutes with an adjustable clock. The study does not establish precise positioning, calibrated gas concentration, certified protective performance or multi-worker identification. These boundaries follow the available implementation and test evidence. They also determine what further measurements are needed before the prototype can be assessed outside a software demonstration.

Figure 1.1 separates what the project undertook from what it did not. The right hand column is not a list of oversights. Each item there needs apparatus the project does not own or a deployment it never had, hence it is being recorded as a boundary and not as an omission, and Chapter 4 returns to every one of them.

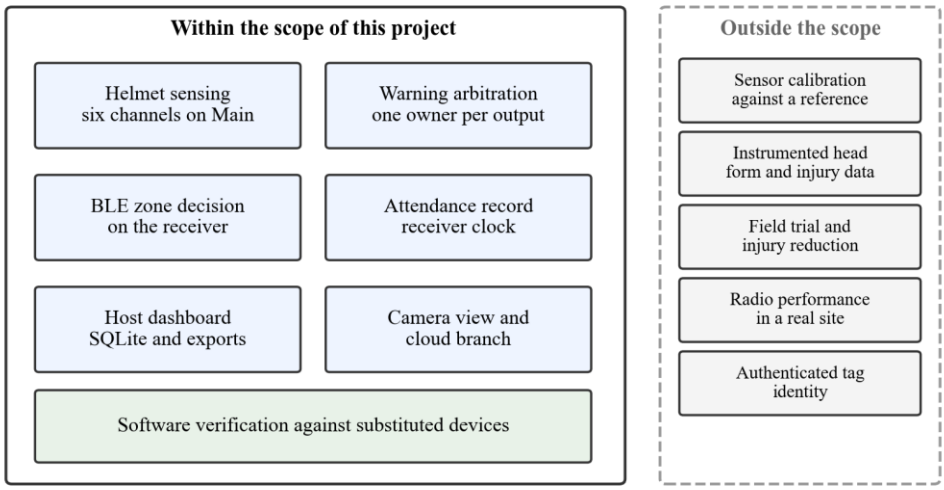


Figure 1.1: Boundary between the work undertaken and the work left outside the project.

1.6 Report Organisation

The report is organised into five chapters. Chapter 1 states the use case, the coordination problem, the objectives and the boundaries of the work. Chapter 2 reviews helmet warning approaches, radio ranging, sensing validity, gas calibration practice, proximity attendance and record keeping, and closes by placing the present implementation against that body of work. Chapter 3 is the longest and carries the requirements, the architecture, the firmware task structure and every sensing and communication algorithm as implemented, together with the power budget and the plan for physical evaluation. Chapter 4 reports what was actually executed, then holds the prepared but unfilled hardware results, the cost and sustainability analyses and the alignment with the Sustainable Development Goals, before setting out the threats to validity. Chapter 5 returns to the objectives and lists what remains. Four appendices carry the operating manual, the source and verification records, the firmware interface tables and the physical test protocols.

CHAPTER 2

LITERATURE REVIEW

2.1 Helmet Warning Approaches

Earlier smart-helmet research provides different ways to decide when a warning is needed. In an underground-mine application, Kim, Baek and Choi (2021) used BLE beacons and helmet-mounted LED indications for personnel proximity warnings. Their work establishes a precedent for an identifiable radio source causing a warning at the helmet. Detection depends on the relevant object carrying a beacon, so this arrangement must be assessed together with the placement and identity of the transmitters. The reported performance belongs to that study and cannot serve as a range measurement for the present receiver.

A later study used an image sensor and artificial intelligence for road-safety warnings. Images from an ESP32-CAM were relayed through a smartphone for cloud object recognition, with visual and tactile warning outputs. Head angle affected the evaluated detection behaviour (Kim and Choi, 2022). The sensing problem is different from beacon recognition: the target need not advertise a BLE identity, but the image must contain information usable by the recognition process. The present Cam program provides streaming and capture, without the object-recognition stage of that study.

These two examples suggest a design comparison based on information requirements. A beacon method asks whether a configured transmitter is detected and

what signal is received. An image method asks what is visible and how it is classified. Neither question is answered merely by adding a camera or radio to a helmet. The implemented processing path determines the capability, and the evaluation must exercise that path. In this project, a camera image is an observation available to the operator. The receiver's BLE measurement is the input used for automatic proximity zoning.

The distinction also affects how a failed warning would be investigated. For the implemented BLE path, the record should establish advertisement reception, zone calculation and command delivery before considering the physical LED or buzzer. For the camera service, a blank view would first require checking capture and transmission. It would be misleading to attribute a missing automatic warning to image-recognition accuracy when the project contains no such recognition function.

The purpose of adopting these studies is to position the selected method, rather than to claim that this prototype outperforms either system. Its contribution is the coordination of local sensor warnings with BLE attendance, camera access and computer records for one helmet. A comparison of accuracy, latency or warning effectiveness would require comparable experimental conditions. No such comparative dataset is available in the supplied archive, so those performance questions remain outside the conclusions drawn here.

2.2 RSSI Ranging and Controller Constraints

RSSI-based proximity requires a model between a radio measurement and a distance estimate. Ramirez et al. (2021) examined BLE RSSI measurement for indoor positioning, including antenna orientation and the experimental layout. Their study makes those conditions part of the measurement record. For the helmet receiver, this supports a test plan that repeats observations at known positions and orientations. It does not provide a calibration for the assembled prototype, whose antenna placement, wearer and room are different experimental conditions.

The receiver's log-distance model uses a reference power and a path-loss exponent. Changing either parameter changes the estimated distance produced by the same RSSI value. Hence a boundary labelled in metres remains conditional on the selected model. Smoothing and hysteresis address different parts of the decision: smoothing changes the input history used in the estimate, when hysteresis changes the threshold needed to leave a previously entered state. Neither operation supplies the missing reference measurements. Their effects on response delay and zone stability should be examined separately.

Radio coexistence is another relevant constraint. Espressif describes the classic ESP32 as sharing a single 2.4 GHz RF module between Wi-Fi and Bluetooth, with time-division use and priority arbitration (Espressif Systems, n.d.b). Placing network activity in another task does not add a second radio to Main. A useful test would observe BLE freshness while HTTP telemetry and cloud requests are active, using the actual firmware configuration. The documentation explains the resource constraint; it does not predict a measured packet delay for this program.

The ESP32 datasheet also documents Wi-Fi, Bluetooth and analogue interfaces, including two 12-bit SAR ADCs (Espressif Systems, n.d.a). These facilities support the selected controller role, but the nominal ADC resolution is not a calibration of the connected battery divider or gas module. Supply conditions, the divider and the software conversion remain part of the measurement chain. A report that quotes the converter resolution as sensor accuracy would combine different quantities.

The selected architecture therefore needs two kinds of checks. Known-distance trials assess how the proximity model behaves in its environment. Concurrent-load trials assess whether the communication paths remain current when the controller serves its other functions. The resulting records should retain configuration and timing so that an observed problem can be associated with a model setting, loss of reception or delayed delivery. Without that distinction, a changing distance display alone gives little evidence about which part of the system requires adjustment.

A different radio would move the same problem somewhere else. Fernandez-Garcia and Gil (2017) built a wearable tracking system on a low-power wide-area network, hence the range and the power budget are being decided by the radio that is chosen long before any distance model is applied to it.

2.3 Sensing and Measurement Validity

Gas sensing illustrates the difference between a responsive component and a validated measurement. The MQ135 manufacturer describes sensitivity to multiple gases and specifies operating and preparation conditions, including an initial preheating period exceeding 48 hours (Zhengzhou Winsen Electronics Technology Co., Ltd., 2021). The present firmware converts two analogue channels into relative percentages. A rising channel can therefore support the programmed warning decision without establishing which gas caused it or what concentration was present.

That distinction affects both naming and evaluation. Calling the displayed quantity a relative gas index preserves the meaning of the implemented conversion. Calling it a calibrated pollutant concentration would require an additional relationship between reference exposure and sensor response. The physical module identity, supply and conditioning would also need to be recorded. The archived program provides no such calibration dataset. Its warning thresholds are consequently assessed as configured software boundaries rather than adopted exposure limits.

A similar distinction applies to the other inputs, although their conversions differ. The battery routine maps divider voltage linearly to a percentage, the FSR routine uses two ADC thresholds for presence, and the motion routine combines acceleration and angular velocity. These choices make each function inspectable. They also expose different verification questions: reference voltage for battery, repeated fit conditions for presence, and labelled movements for impact. A single demonstration in which the indicators change cannot answer all three questions.

The design review therefore favours traceable decisions over a common accuracy claim. Each reported output should be linked to its input, transformation and state rule. The methodology supplies those links for the implemented modules, while the results chapter identifies which software paths were actually executed. This avoids aggregating unlike evidence, such as an export test and a sensor demonstration, into an unsupported overall reliability percentage.

Across the reviewed work and the current source, the unresolved issue is coordination under real operating conditions. Beacon warnings require radio reception, camera observation requires usable images, and local sensing requires an appropriate measurement chain. This prototype connects those functions through shared indications and a desktop record. Its engineering value can be discussed through that architecture and its tested software behaviour. Claims about workplace protection, positioning accuracy or comparative superiority would need a separate evaluation with explicit conditions and repeatable observations. The literature informs that evaluation plan without supplying results for the present hardware.

2.4 Gas Sensing and Calibration Limits

Metal oxide semiconductor gas sensors are the cheapest practical way to obtain a response to combustible and toxic gases, and the MQ series used in this implementation belongs to that family. The sensing element is a heated tin dioxide layer whose surface conductivity changes when a reducing gas adsorbs onto it. The device therefore reports a resistance, not a concentration. Converting that resistance into parts per million requires a calibration curve taken against a known gas mixture at a known temperature and humidity, and the curve is specific to the individual element.

The literature is consistent about where this becomes a problem. Sensor arrays of this type drift over their service life, and the drift is large enough that dedicated compensation methods are being developed to keep a trained classifier usable across months (Jiang et al., 2021). Drift is not the only difficulty. The same surface chemistry

that responds to the target gas also responds to humidity and to ambient temperature, hence a reading that rises during a humid afternoon cannot be attributed to gas without a controlled comparison.

This matters directly for the present work. The firmware does not attempt a ppm conversion at all. It maps the raw twelve bit ADC count onto a percentage between a baseline constant and a raw maximum constant, and those two constants are compile time values that were not obtained from a calibration gas. What the percentage represents is a position between two assumed endpoints. Which is an honest indicator of relative change, and is not a concentration measurement.

Reporting an uncalibrated index as though it were a concentration would be the common failure in student projects of this kind, hence the decision here to keep the scale explicitly dimensionless and to name it a percentage of span throughout the report. The classification boundaries at 25, 51 and 76 percent are design choices that set how early the wearer is warned. They are not exposure limits and they are not being presented as such. An occupational exposure limit is a legal quantity tied to a measured concentration over an averaging period, and nothing in this implementation produces that quantity.

There is a further constraint from the controller. Both gas channels sit on ADC1 of the ESP32, and successive approximation converters of this class carry integral and differential nonlinearity that varies across the input range (Michaeli, Michalko and Šaliga, 2008). The firmware averages ten samples per channel per cycle, which suppresses random noise but will not remove a systematic nonlinearity. This causes the percentage to carry an error that averaging cannot reach.

2.5 Proximity Attendance and Identity Assurance

Attendance recorded from a radio tag is a different problem from attendance recorded by a person. The receiver never sees an identity in the cryptographic sense. It sees an

advertising packet carrying a device name and a service identifier, and it infers presence from the fact that the packet arrived with a usable signal strength. Everything downstream inherits that assumption.

The weakness has been demonstrated rather than only argued. Kim, Lee and Paek (2018) built a signal imitation attack against a beacon based electronic attendance system and showed that an attacker able to replay or imitate the advertised identifier can register attendance without being present. The implementation described in this report carries the same exposure. Its tag name is the fixed string TAG001, the service identifier is a compile time constant, and no pairing, bonding or challenge is performed before the receiver accepts a sighting. A second device advertising the same name and service would be treated as the helmet.

That is a real limitation and it is being recorded here rather than deferred to the conclusions. The mitigation is standard and is not implemented, which would be BLE bonding with a resolvable private address, so that the receiver verifies a key instead of a string.

Dwell time is the one defence that is present. The receiver requires the tag to remain inside the boundary for a configurable dwell period before a check in is written, and it requires an absence of five seconds before a check out. A brief imitation as somebody walks past the door will not by itself produce a row, when a sustained imitation will. The dwell control therefore raises the effort required and does not remove the attack.

The wider positioning literature supports the choice of a boundary decision over a coordinate estimate. Surveys of radio based indoor positioning describe accuracy that depends on the environment, the radio technology and the density of anchors, and single anchor received signal strength is at the weak end of that range (Subedi and Pyun, 2020). Alternative carriers exist. A low power wide area network gives far greater range for a wearable tracker at the cost of update rate and of infrastructure (Fernández-Garcia and Gil, 2017). For a single room deployment with

one receiver, none of these alternatives is being claimed as tested; the design simply avoids reporting a position it cannot support and reports a zone instead.

The mitigation named above has a literature behind it. Address privacy in Bluetooth Low Energy rests on resolvable private addresses, which let a bonded peer recognise a device while an eavesdropper cannot link its appearances, and the mechanism and its weaknesses have been analysed formally (Sun and Tian, 2022). Adopting it here would replace a fixed advertised string with a key the receiver can verify. Which is the single change that would make an attendance row mean what it appears to mean.

2.6 Helmet Compliance Detection and Presence Sensing

A separate line of research treats the helmet as something to be observed rather than instrumented. Deep learning detectors trained on site imagery decide whether a worker is wearing a helmet at all, and multi scale feature methods have been applied to that problem with good reported accuracy on construction site datasets (Han and Zeng, 2022). The distinction from the present work is worth drawing clearly, since both approaches are being called smart helmet research in the literature.

A vision detector answers compliance. It tells a supervisor that somebody is not wearing the helmet, from a camera mounted on the site and looking outward. An instrumented helmet answers condition. It tells the wearer that the air around their head has changed, from sensors mounted on the helmet and looking at the wearer. The two are complementary and neither substitutes for the other, when a great deal of published work treats them as competing.

This project sits on the instrumented side and its presence detection is deliberately crude. A force sensitive resistor under the harness reports a raw count, and two thresholds decide worn or removed with a gap between them so the state does not flicker while a wearer adjusts the fit. A force sensitive resistor is not a precision

instrument. Its response is nonlinear, it drifts with sustained load, and its output depends on how the force is distributed across the sensing area, hence the design uses it only for a binary decision and never reports a force.

The camera in this system is closer to the vision literature in hardware and much further from it in function. It streams JPEG frames over HTTP and performs no recognition of any kind. No detection model runs anywhere in the implementation, and the report does not claim any. What the camera provides is a view that an operator can open, which is useful for confirming a situation after an alert and is not an automated check.

Kim and Choi (2022) relayed camera images through a smartphone to a cloud recogniser and reported detection performance for that arrangement. Their result belongs to their architecture. It cannot be transferred to this implementation, which has no recogniser, and it is cited here only to mark where the present work stops.

2.7 Local Records, Cloud Storage and Security

The final part of the review concerns what happens to a reading after it has been taken. A monitoring project that keeps its evidence only in volatile memory has produced a display and not a record, hence the attention given here to storage and to the paths by which storage can be corrupted or lost.

Two storage locations exist in this system and they fail differently. The local database on the host computer is a single SQLite file opened in write ahead logging mode, which allows a reader and a writer to proceed without blocking each other and keeps a committed row durable across an unexpected shutdown. The cloud branch is a Firebase Realtime Database tree written over HTTPS. The local file is authoritative in this design. If the network is unavailable the events remain in the database with their synchronised flag unset, and the uploader will retry them later, when a design that wrote only to the cloud would simply lose the interval.

Duplicate suppression is the part most often left out. Events arriving from the receiver carry a boot identifier and a monotonically increasing sequence number, and the host composes a primary key from both. A replayed poll therefore inserts nothing, and a receiver that has rebooted produces a new boot identifier so that its sequence restarting at one does not collide with the earlier session. This is being tested directly in the supplied test suite.

Export introduces a security concern that is easy to miss in an engineering report. Attendance rows contain operator supplied text such as a worker name, and a spreadsheet will interpret a cell beginning with an equals sign as a formula when the file is opened. The storage module prefixes such a cell with an apostrophe before writing it. Which prevents a value in the register from executing as a formula in the reviewer's spreadsheet.

Transport security is weaker on the firmware side and the report will not present it as adequate. The Main program opens its TLS connection to Firebase with certificate verification disabled, marked in the source as a prototype measure. The consequence is that the helmet will accept any certificate presented by whatever host answers at that address, hence removing the guarantee that the endpoint is genuine. Automated tooling for finding exactly this class of defect in Wi-Fi connected devices exists because the defect is common (Chen, Yu and Li, 2025). The correct fix is to pin or to bundle a root certificate, and it has not been applied.

2.8 Position of the Present Implementation

Table 2.1 compares the information used by the reviewed warning approaches with the present implementation. The comparison concerns sensing and processing responsibilities. It deliberately leaves performance values out because the studies and this prototype do not share an experimental dataset. This makes the design difference visible without implying a ranking that the evidence cannot support.

Figure 2.1 groups the reviewed work by the question each study answers. The four columns are not competing schools and a study in one column is not a weaker version of a study in another, since warning a wearer, observing compliance, locating a worker and trusting a record are separate problems that happen to share a helmet. What the figure shows is that the reviewed work answers them one or two at a time. This implementation attempts all four at once, which is its only real claim to novelty, and it does so with evidence for the software paths alone.

Table 2.1: Comparison of Warning Approaches

Source	Information / processing	Relationship to this project
Kim, Baek and Choi (2021)	BLE beacons; helmet LED warning	Comparable radio identity principle; their range is not this receiver's range.
Kim and Choi (2022)	ESP32-CAM images; cloud recognition	Cam here streams and captures; no recognition stage.
Ramirez et al. (2021)	Indoor BLE RSSI measurements	Supports recording distance, orientation and layout during calibration.
Main.zip implementation	Local sensors; BLE zones; attendance; computer records	Source integration and selected host tests; no comparative field dataset.

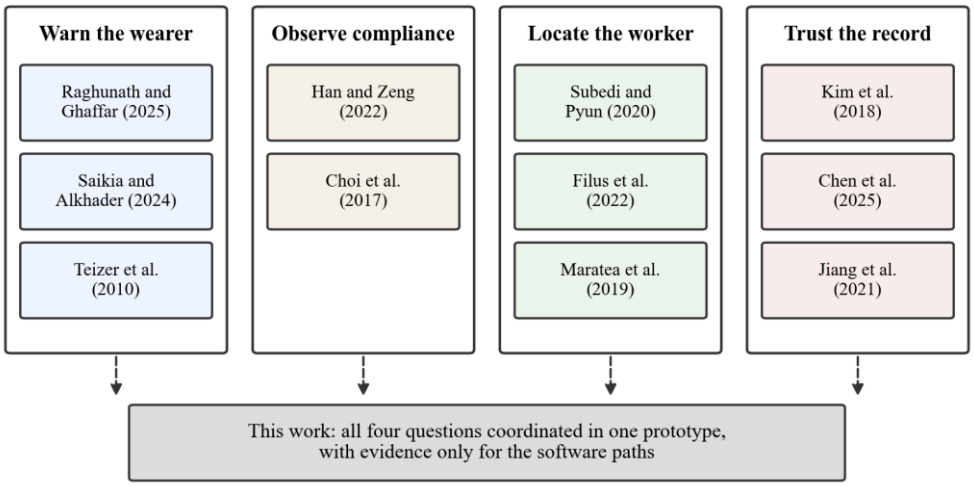


Figure 2.1: The reviewed work grouped by the question each study answers.

2.9 Environmental Temperature and Heat Stress Interpretation

Heat is an ordinary working condition on a site here, and it is one reason temperature sensing keeps appearing in helmet research. A long shift in a hot place will reduce concentration and will raise tiredness, and at the far end of it the worker is being exposed to heat illness. Sharma, Suri and Kant (2022) measured occupational heat stress with a sensor-based helmet and read the result against environmental ergonomics, which shows a helmet interpreting a thermal risk and not merely displaying a number. Campero-Jurado et al. (2020) placed temperature and humidity inside a wider risk-detection helmet, hence the surrounding condition is treated there as part of the safety assessment itself.

Raghunath and Ghaffar (2025) sit closest to the present work. Their IoT-enabled helmet carried a temperature-humidity sensor together with a motion sensor, which is the same pairing used here. Two points from that study matter for this project. The first is that cheap and easily obtained parts still carry a usable safety function in a prototype, when the second is that air temperature alone was not trusted as the indicator, because they computed a heat index through the Rothfusz regression, which is published by the U.S. National Weather Service (2022), so that humidity enters the result. A worker does not feel the dry bulb value directly. Humidity will slow the cooling that sweating provides.

One fixed temperature is a weak decision rule and the literature says so. Raghunath and Ghaffar (2025) read their computed heat index against a Heat Stress Matrix adapted from Dubai Municipality (2019), hence the risk is being graded across bands instead of switched at a single point. Sharma, Suri and Kant (2022) reached the same position from the working context. The present study follows it, where ambient temperature and relative humidity are both measured and the heat index computed from the pair is the thermal indicator that the rest of the system reads.

The DHT22 suits the assigned function because it returns the two quantities the heat index needs, and it is cheap, small and simple to wire to an ESP32. The

subsystem is hence a practical thermal indicator inside the helmet and not an occupational heat-stress diagnosis.

Temperature data is worth more when something follows from it. Raghunath and Ghaffar (2025) tied a high heat index to drinking water, to shaded rest and to shorter exposure time. A reading that reaches no decision is only a number on a screen, and the same idea is applied here, where the measured condition is sorted into risk bands that the system will use for warning and for logging, and a supervisor will later read those same bands.

Reading a thermal risk from an index is not a new practice either. Ramanathan and Belding (1973) evaluated the wet bulb globe temperature index physiologically for occupational heat stress, which is the same idea this subsystem applies with a cheaper sensor and a simpler index.

2.10 Impact and Motion Sensing in Helmet Systems

Impact detection is another important function in smart helmet research because sudden falls, collisions, or unusual head movements may indicate that a serious accident has happened. Hence, the worker may need immediate emergency medical attention. Lee et al. (2022) have found that inertial measurement units are one of the most generally used sensor types in industrial smart helmets, particularly for accident as well as activity monitoring. Raghunath and Ghaffar (2025) also used an MPU6050 gyroscope-and-accelerometer module in their worker-safety helmet. Their system analysed motion patterns to trigger fall alerts, which supports the use of a low-cost 6-axis sensor in construction-related wearable safety systems.

Other studies also support the use of motion sensing in helmet-based safety applications. Kim et al. (2018) have shown that a three-axis accelerometer is able to help recognise helmet-wearing conditions during construction activities. Although their study was not focused on fall detection directly, it still shows that motion-based

helmet sensing remains practically useful. Marquez-Sanchez et al. (2021) also included accelerometers and shock-related sensing in a smart-PPE platform for workplace safety. Their work suggests that motion data can be used not only to detect individual accident events, but also to support anomaly detection and provide a better understanding of the safety condition of the worker. These studies show that inertial sensing is a reasonable and technically supported component in smart helmet systems.

More practical studies show that many low-cost smart helmet prototypes use threshold-based logic to detect accidents or falls. For instance, Raghunath and Ghaffar (2025) have applied a resultant-acceleration threshold of 2.5 g, followed by rapid deceleration within a short period of time, in order to detect fall-like events. Besides, they also emphasised the need to balance sensitivity and false alarms. Minhas et al. (2026) have also described a low-cost smart helmet with accident detection and emergency response, which shows that embedded event-triggered safety functions can be applied in wearable helmets. Swetha and Devigasri (2025) also discussed impact detection in an industrial smart helmet, where head-impact interpretation was linked to immediate alerting. Although these systems are not medical diagnostic devices, they show that threshold-based motion interpretation is practical for embedded safety prototypes.

However, the literature also makes it clear that low-cost impact detection has some limitations. The sensor readings can be affected by where the sensor is mounted, the posture of the worker, bending, jumping, vibration, or other strong movements. Some of these actions may produce signals that look similar to a real impact. Raghunath and Ghaffar (2025) selected their threshold by considering both sensitivity and specificity, which shows that the threshold is mainly an engineering trigger and not a confirmed injury indicator. In the present study the MPU6050 subsystem is hence used as an early-warning mechanism for possible impact or abnormal motion. It is not intended to diagnose concussion, trauma, or any medical condition.

The choice of MPU6050 is practical for the present project. This module combines a three-axis accelerometer and a three-axis gyroscope in one compact device. This reduces wiring complexity while still providing both linear and rotational motion

data. Compared with using only an accelerometer, the MPU6050 is more suitable for observing sudden movement patterns on a wearable helmet. This is useful because the aim of the subsystem is not only to measure simple tilt or orientation, but also to identify suspicious movement that may suggest an accident or unsafe condition.

Additionally, previous studies also showed that careful selection of motion features is needed for reducing false alarms if using inertial sensing alone for impact detection. For the embedded impact detection subsystem, using both acceleration magnitude and rotational motion is better than relying only on one axis or one sensor output. Since the MPU6050 can provide three-axis acceleration and three-axis gyroscope data in one module, this makes it appropriate for the present project. Hence, this subsystem applies a two-stage checking approach, in which the abnormal resultant acceleration will be used as the first trigger, while the abnormal gyroscope magnitude is used to verify if the motion may be related to an impact event. Although this approach is still much simpler than machine-learning-based classification, it is more reliable than just using a single-axis threshold.

The sensing element does not have to be inertial at all. Saikia and Alkhader (2024) measured head injury with a smart textile impact sensor built into an e-helmet, when the present subsystem stays with the MPU6050 because the same module already carries the rotational channel the confirmation stage needs.

2.11 Helmet-Mounted Vision and Remote Assistance

A camera changes what a supervisor is able to know about a site, because a reading gives a value when a picture gives the scene in front of the worker. Merchan-Cruz et al. (2025) reviewed smart helmets with integrated vision and found helmet-mounted cameras being used for infrastructure inspection and for remote support, and in hazardous or complex workplaces for situational awareness as well. Visual information improves the inspection and the understanding of the workplace in their review.

Abdullah, Celik and Celik (2025) proposed an affordable camera-based safety and quality framework for construction work in developing countries, hence the cost of the camera is being treated there as part of the engineering problem and not as an afterthought. Wu et al. (2023) monitored helmet and harness compliance from remote video and showed the stream supporting supervision in a real working environment. Neither of them is an ESP32-CAM design. What the two studies support is the narrower claim made here, which is that a video channel works beside the sensor channels by showing what the numbers cannot describe.

Vision systems are not one class of thing. Merchan-Cruz et al. (2025) describe VSLAM, multi-sensor fusion and edge-cloud processing at one end of the range, when their review also names battery life, processing power and connectivity as the limits a wearable runs into. A prototype that only streams live video sits at the other end of that range, and this project stays there deliberately, hence no recognition, no mapping and no augmented reality is being attempted on the board.

Scope decides it as well. The camera is one sub-function among several safety and IoT features, so the ESP32-CAM is treated as a focused remote-visualisation module. It provides a low-cost forward-facing feed, which lets a remote expert look at the surroundings of the worker or check an event that another subsystem has already raised.

CHAPTER 3

METHODOLOGY AND WORK PLAN

3.1 Method and Work Plan

The work plan separates the available software evidence from the measurements required on the assembled prototype. Table 3.1 lists the stages and their deliverables without assigning retrospective dates to work for which no dated record is available. Source inspection and the recorded Python execution are complete for this report. Firmware compilation, wiring inspection and physical evaluation remain separate stages. Their proposed procedures are included so that later results can be compared with a defined method rather than with an informal demonstration.

Figure 3.1 shows the same work plan as a schedule. The bars overlap heavily because firmware for one subsystem was written while another was being debugged, which is normal for a single developer and is also why no week can be attributed cleanly to one deliverable. The hatched bar past week twenty is the calibration and trial work that has not been carried out.

Table 3.1: Work Plan and Evidence Status

Stage	Output	Status in this report
1. Source and format audit	Archive map, source ledger, V11 format contract	Completed inspection
2. Software verification	Unittest execution log; assertion mapping	28 tests passed, 16 September 2026
3. Build and wiring baseline	Three build logs; board and wiring record	Proposed; final build evidence absent

Stage	Output	Status in this report
4. Sensor and radio evaluation	Reference readings; labelled motion and BLE trials	Proposed; no measured dataset
5. Combined operation	Output timing; event counts; camera/cloud observations	Proposed; no physical result
6. Report and review	Source-grounded report and supporting records	Documented in this submission

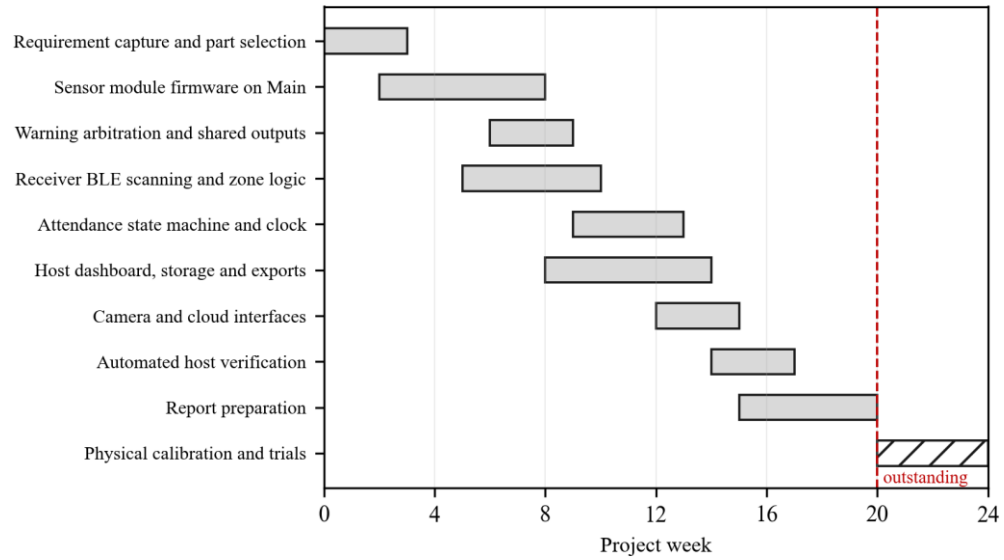


Figure 3.1: Work plan as a schedule, with the outstanding calibration and trials hatched.

3.2 Requirements and Design Constraints

Before any component was selected the work was bounded by a small set of requirements, and those requirements are stated here so that Chapter 4 can be read against them rather than against an impression. They were derived from the objectives in Section 1.4 and from what a single student can verify without a laboratory gas rig.

The functional requirements are as follows. The helmet must sense ambient light, two gas channels, head motion, temperature with humidity, battery state and whether the helmet is being worn, and it must do so without any one sensor blocking another. It must produce a warning that the wearer can perceive while wearing the

helmet, which restricts the output to light and sound. It must expose its readings to a computer over the local network. A separate receiver must decide whether the helmet is inside a boundary and must own the clock used to stamp attendance. The computer must hold the record and must be able to export it.

The non functional requirements are tighter and they shaped more of the design. No task may use a blocking delay long enough to delay another, hence the cooperative interval structure described in Section 3.2. Every physical output is owned by exactly one arbiter, because two modules writing the same pin will produce a warning that means nothing. A reading that cannot be trusted must appear as null and not as zero, since a zero lux value and a disconnected light sensor are different situations and a chart cannot tell them apart afterwards. The record must survive a restart of any of the four programs.

Several constraints were fixed and could not be traded. The ESP32 exposes ADC2 to the Wi-Fi radio, which means the analogue channels had to be placed on ADC1, and that decision then fixed the pin map for the gas, battery and presence inputs. The BLE stack and the Wi-Fi stack share one radio, hence the receiver's scan window and its HTTP service compete for time. Cost was capped at the level of parts a student buys individually. Lastly the project had no access to certified calibration gas, no environmental chamber and no instrumented head form, which is the constraint that decides what Chapter 4 is allowed to claim.

Table 3.2 records the requirements against the section where each is implemented and the evidence class that supports it. The evidence class is important. A requirement satisfied in source and exercised by an automated test is in a different position from a requirement that is implemented but has never been run against real hardware, and the table keeps the two apart.

Table 3.2: Project Requirements and Their Evidence Class

Ref	Requirement	Implemented in	Evidence class
F1	Acquire six sensor channels without mutual blocking	Sections 3.5 to 3.9	Source

Ref	Requirement	Implemented in	Evidence class
F2	Warn the wearer through light and sound only	Section 3.10	Source
F3	Expose helmet readings to the host over the local network	Section 3.16	Source and test
F4	Decide a proximity zone on a separate receiver	Sections 3.11 and 3.12	Source
F5	Stamp attendance from a receiver-owned clock	Section 3.13	Source and test
F6	Hold the record locally and export it	Section 3.16	Test
N1	No blocking delay in any acquisition path	Section 3.4	Source
N2	Exactly one arbiter owns each physical output	Section 3.10	Source
N3	An untrusted reading is reported as null and never as zero	Section 3.16	Source and test
N4	The record survives a restart of any program	Section 3.16	Test
C1	Analogue inputs confined to ADC1 while Wi-Fi is active	Section 3.3	Source
C2	No certified calibration gas, chamber or head form	Section 4.5	Not satisfiable

3.3 System Architecture

Figure 3.2 shows the implemented communication paths. Receiver is responsible for the proximity decision, while Main owns the local sensor outputs. The computer reads both devices because their information serves different purposes. Table 3.3 records the Main pin assignments and scheduled acquisition intervals used in the source. The intervals describe program settings; execution time and competing tasks can affect when an operation actually completes.

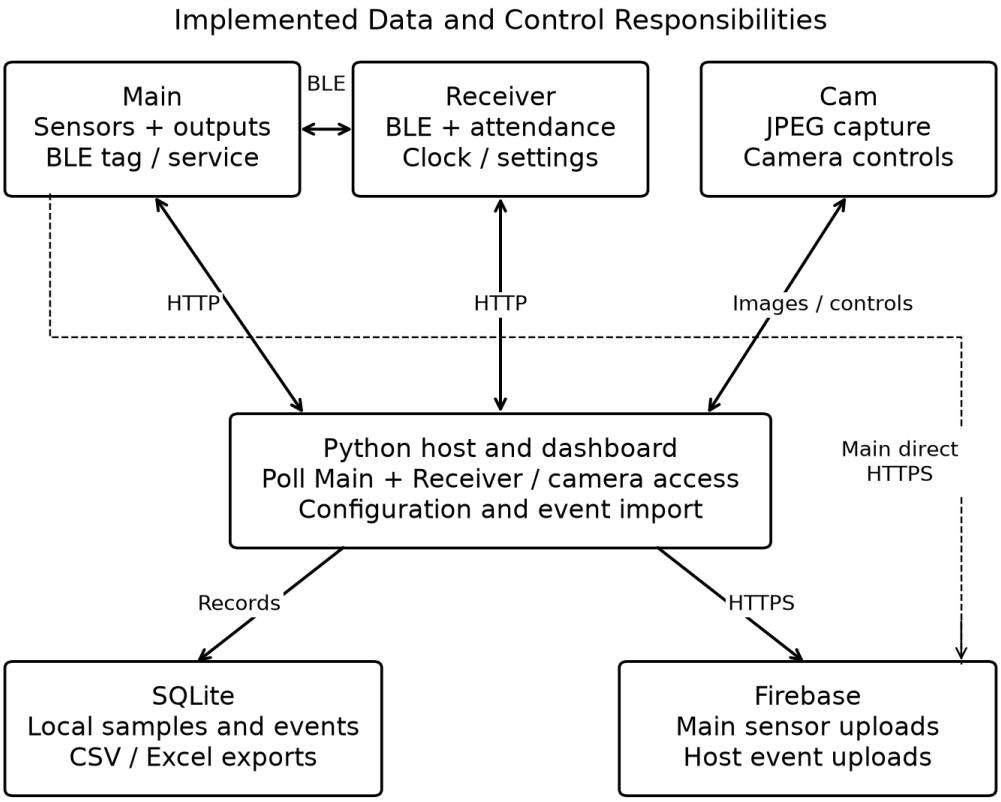


Figure 3.2: Data and control paths as they are implemented across the four programs.

Table 3.3: Main Pin Assignments and Program Settings

Function	GPIO	Configured operation
BH1750 / MPU6050	21 SDA; 22 SCL	I2C 100 kHz; light 1 s; motion 50 ms
Lamp	25	PWM 5 kHz, 8 bits, upper command 178
MQ-7 / MQ-135	34 / 35	500 ms; 10 readings per channel
Yellow / red indicator	18 / 19	Shared-output arbitration
Shared buzzer	33	Active high; ON for gas danger, confirmed impact, temperature Danger or a red zone
DHT22	4	2 s
Battery divider	36	5 s; 20 readings
FSR402	39	200 ; 10 readings

3.4 Firmware Task Structure and Timing

The Main program runs six acquisition modules, a BLE server, an HTTP server and a cloud uploader on one dual core controller, and the way that work is divided decides whether a warning arrives late. Two mechanisms are being used together.

The first is a cooperative interval structure inside the main loop. Every module owns a timestamp of its last execution and an interval constant, and each call returns immediately if the interval has not elapsed. Light is read every 1000 ms, gas every 500 ms, motion every 50 ms, temperature every 2000 ms, battery every 5000 ms and presence every 200 ms. No module calls a blocking delay, hence the loop completes quickly and the slowest module cannot starve the fastest. The only delay in the loop is a single millisecond yield that lets the scheduler run.

The second is a separate FreeRTOS task for the network. Uploading to Firebase involves a TLS handshake and a request that can take seconds when the link is poor, and running that inside the loop would stall impact detection for the duration. The uploader is therefore pinned to its own task, and it communicates with the acquisition modules only through volatile variables and one critical section that guards the impact event triple. A failed upload delays nothing except the next upload.

The receiver uses the same idea with different boundaries. Its BLE scan and state evaluation run on a dedicated worker task pinned to core 0, when its HTTP service runs from the main loop, and a mutex protects the shared state so that a web request never reads a half written structure. The worker scans for one second, evaluates, attempts one connection to write the command characteristic, and yields for 200 ms.

There is a cost to this arrangement that the report should state. Values crossing between the loop and the network task are marked volatile but are not, with the exception of the impact triple, read under a lock. A snapshot assembled for an upload can therefore contain a temperature from one instant and a gas percentage from the next. For a ten second telemetry cycle this skew is unimportant. For a claim about the

exact simultaneity of two readings it would not be, hence the report does not make such a claim anywhere.

A second cost is more visible. The latest impact slot holds only the most recent event, so two impacts occurring inside one ten second polling interval will be delivered to the host as one, with the sequence number advancing by two. The host counts the sequence and not the payload, which limits the loss to the intermediate magnitudes and not to the fact that events occurred.

Figure 3.3 shows one complete acquisition cycle across the four programs, with the two device reads issued together and the storage and cloud writes that follow them. The order in the diagram is the order the host performs them, hence a failure at any one arrow leaves the later steps to the next cycle instead of aborting the whole pass. Table 3.4 collects the interval and the sample count that each channel is being read at, so the timing above can be checked line by line.

Table 3.4: Acquisition Intervals and Sample Counts in Main

Module	Interval	Samples per cycle	Blocking
Ambient light (BH1750)	1000 ms	1	None
Gas (MQ-7 and MQ-135)	500 ms	10 per channel	200 us between samples
Motion (MPU6050)	50 ms	1	None
Temperature and humidity (DHT22)	2000 ms	1	Library read
Battery divider	5000 ms	20	500 us between samples
Helmet presence (FSR402)	200 ms	10	None
Output arbitration	Every loop pass	Not applicable	None
Firebase current and history	10000 ms	Not applicable	Separate FreeRTOS task

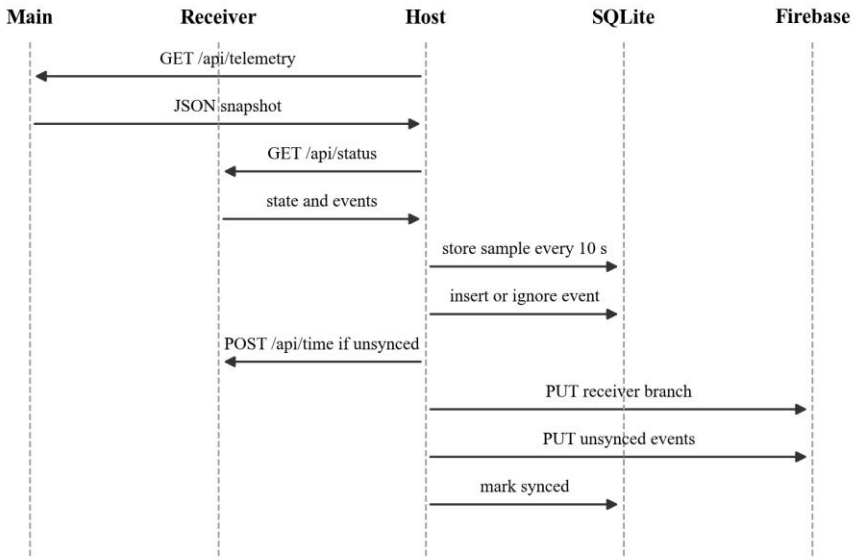


Figure 3.3: Sequence of one host acquisition cycle across the four programs.

3.5 Acquisition and Illumination

The six acquisition modules share Main's Arduino loop. Calling a module does not necessarily take another measurement, because each routine first checks how much time has elapsed since its preceding update. Motion has the shortest configured interval, 50 ms. Gas, light, temperature, battery and presence readings follow their own intervals. The two MQ channels count as one gas module in this arrangement. Firebase requests are handled separately on core 0 by a FreeRTOS task with priority 1 and a 16,384-byte stack allocation. Keeping HTTPS requests out of the acquisition loop removes a direct wait for each transaction; it does not establish a measured sampling rate under load.

Both I2C devices use SDA 21 and SCL 22 at 100 kHz. Startup includes the MPU6050 library's gyroscope-offset calculation, so the helmet must remain stationary during that procedure. Main initiates Wi-Fi association without waiting for the router before starting the subsequent sensor processing. The network is still needed to obtain the HTTP telemetry and send cloud records.

Illumination is read from the BH1750 once per second in continuous high-resolution mode. The lamp on GPIO 25 receives a 5 kHz PWM signal with eight-bit resolution. Here the upper command is 178 rather than 255. That distinction matters when interpreting the displayed brightness: 178 counts corresponds to approximately 69.8% of the available PWM range, even though it is the highest command permitted by the routine. A reading of 20 lux or less selects this limit. The command falls to zero by 150 lux. Between them, the code forms $x = (L - 20)/130$ and calculates $\text{round}[178(1 - x)^2]$. The squared term gives the intervening response a curved shape. If the sensor returns a negative reading, its validity flag is cleared and the previous physical PWM setting remains.

Figure 3.4 plots the commanded duty against illuminance for the implemented mapping. The curve is computed from the source and is not a measurement. Note how shallow it becomes above roughly 100 lx, which means the lamp is being turned down well before the room is bright.

Equation 3.1 normalises the measured illuminance onto the interval between the two clamps, and Equation 3.2 then maps that normalised position onto a duty command through an inverse square response. The squaring is what keeps the lamp bright across most of the usable range instead of fading linearly.

$$L_{norm} = (E - E_{min}) / (E_{max} - E_{min}) \tag{3.1}$$

$$PWM = \text{round}[\, D_{max} \times (1 - L_{norm})^2 \,] \tag{3.2}$$

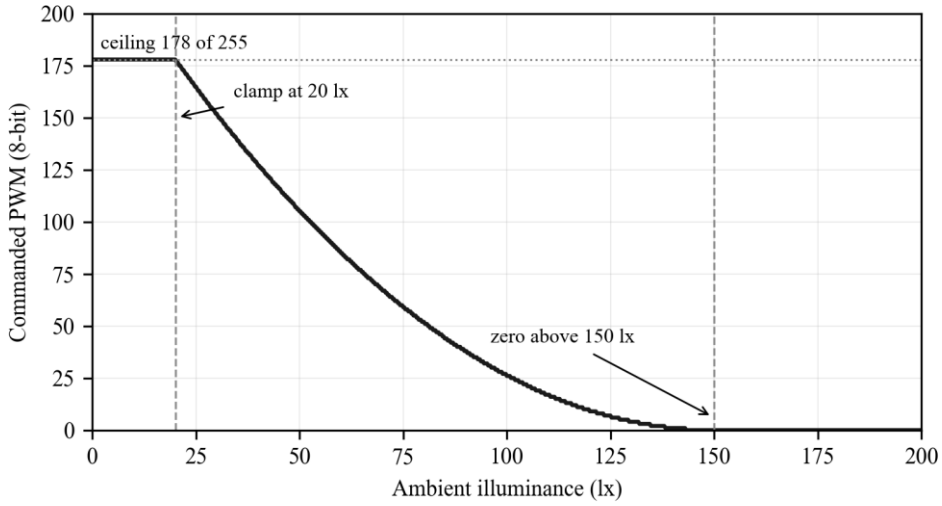


Figure 3.4: Commanded lamp PWM against ambient illuminance, computed from the implemented mapping.

3.6 Gas Processing

Gas processing takes ten readings per channel every 500 ms, with 200 µs between samples. MQ-7 uses GPIO 34 and MQ-135 uses GPIO 35. Their baseline and upper ADC counts are respectively 600/3,200 and 700/3,300. After averaging, integer arithmetic applies p = clamp[100(r − b)/(rmax − b), 0, 100]. The larger of the two resulting indices supplies the combined warning value. Hence an elevated reading from either channel is sufficient to raise that value, without requiring agreement between the sensors. The conversion describes a relative input scale; it contains no gas-specific concentration calculation.

Gas warning levels must be read in the units used by Main. An index below 25% is SAFE; reaching 25%, 51% and 76% selects WARNING, HIGH WARNING and DANGER respectively. The calculation starts with ADC counts, and no subsequent step converts the result to parts per million. Voltage constants elsewhere in the program do not change this fact. A gas-specific calibration would need a reference exposure and a suitable response model before these displayed percentages could carry a concentration meaning.

Equation 3.3 states the mapping actually performed on each gas channel. Neither constant in it was measured.

$$G_{\%} = 100 \times (N - N_{\text{base}}) / (N_{\text{max}} - N_{\text{base}}) \quad (3.3)$$

3.7 Motion Detection

The motion rule uses two resultants calculated from the MPU6050 axes. For each resultant, the three axis values are squared, added and square-rooted. The acceleration result is expressed in g and the gyroscope result in degrees per second. Acceleration above 2.5 g starts a candidate. Confirmation then requires angular velocity above 250 degrees per second. A candidate that remains unconfirmed expires after 300 ms, but the order of comparisons deserves attention: Main checks the gyroscope first, including at the first sample beyond that interval. A boundary test must reproduce this ordering instead of assuming that expiry is checked first.

When confirmation occurs, Main retains the values from that sample and increments its event sequence. These are confirmation values, so a peak occurring earlier in the movement may not appear in the stored record. The record itself holds only the latest event. If collection is delayed across several impacts, a later record can replace one the computer has not yet obtained.

An alert lasts 1,500 ms. The separate 1,000 ms cooldown is measured from its trigger, not from its end. Since the active alert prevents re-triggering for the longer interval, describing the cooldown as an extra second after the alert would overstate the delay implemented by the program.

Figure 3.5 shows the states and the transitions between them. A candidate that is never confirmed returns to idle without producing any record at all, hence a knock that shakes the helmet without rotating it will not be reported as an impact.

The resultant quantities compared against the thresholds are given by Equation 3.4 and Equation 3.5. Taking the magnitude of the three axis vector removes any dependence on how the sensor happens to be oriented inside the shell, which matters because the module is glued in by hand.

$$a_{res} = \sqrt{(a_x^2 + a_y^2 + a_z^2)} \tag{3.4}$$

$$\omega_{res} = \sqrt{(\omega_x^2 + \omega_y^2 + \omega_z^2)} \tag{3.5}$$

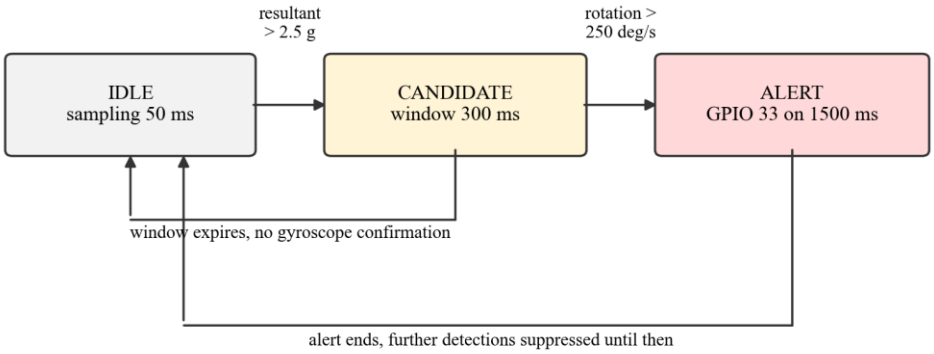


Figure 3.5: States and transitions of the two-stage impact detector.

3.8 Temperature and Heat Index

The DHT22 measurement path runs every two seconds on GPIO 4. Each reading is checked before it is used for heat-index calculation, while invalid or missing readings are ignored until the next 2-second sampling cycle. For a valid pair, Main converts temperature to Fahrenheit, applies its heat-index polynomial and converts the answer to Celsius. It does not first restrict the inputs to a documented validity domain or apply a humidity correction. The configured categories are Normal up to 34 °C, Caution above 34 °C through 39 °C, Warning above 39 °C through 45 °C and Danger above 45 °C.

Checking that arithmetic alone would leave the sensor input unverified. A useful thermal test must compare the measured inputs with a reference as well as

check the calculated index, because an input error can pass through a correctly implemented equation.

Figure 3.6 plots the computed index against air temperature at four humidities, with the firmware boundaries drawn across it. The spread between the curves is the point of the calculation, since 34 C at 40% humidity and 34 C at 95% humidity place the wearer in different bands entirely.

Equation 3.10 states the conversion applied around the regression. The regression itself is a nine term polynomial in Fahrenheit temperature and relative humidity, and it is reproduced exactly as implemented in Listing E.4 rather than retyped here, since a transcription error in a coefficient would be invisible in the prose and would change every reported value.

One point about the index itself should be recorded before it is used to warn anyone. The heat index is a comfort derived quantity and not a physiological measurement, when what actually matters to a worker is whether the body can still shed the heat it produces. Recent work comparing several thermal indices found that the heat index and the humidex track heat stress compensability during moderate work, where several other common measures did not (Zhao, McDermott and Vanos, 2025). That supports the choice of index here. It does not make the boundaries at 34, 39 and 45 C physiologically validated, since those were carried over from the source and were never fitted to any worker.

The present study uses a heat stress matrix that is adapted from Dubai Municipality (2019), which was also used by Raghunath and Ghaffar (2025), instead of relying only on one fixed temperature value. For the software implementation, the matrix is simplified into four status levels based on the calculated heat index: Normal when the heat index is 34 degrees Celsius or below, Caution above 34 up to 39 degrees Celsius, Warning above 39 up to 45 degrees Celsius, and Danger above 45 degrees Celsius.

$$\mathrm{HI_C} = (f_{\mathrm{Rothfusz}}(T_{\mathrm{F}}, \mathrm{RH}) - 32) \times 5 / 9 \tag{3.10}$$

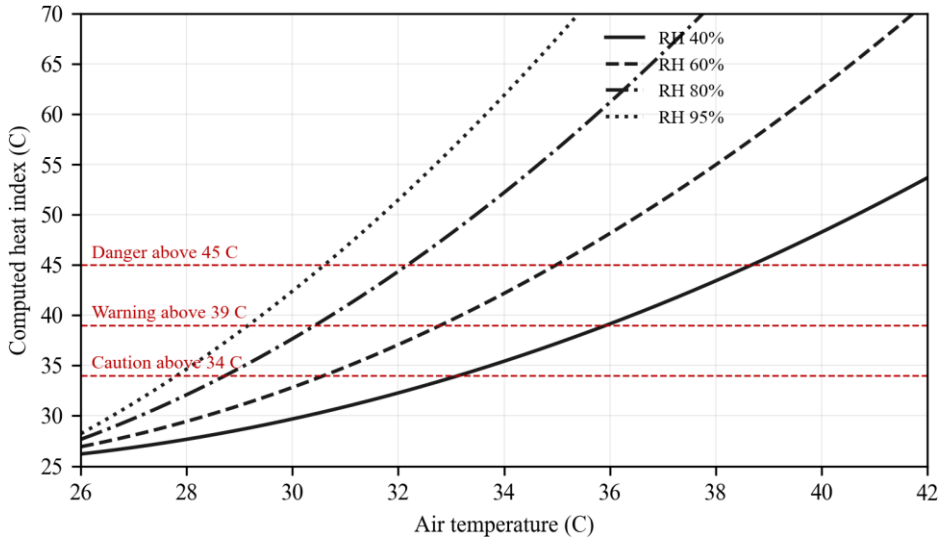


Figure 3.6: Computed heat index against air temperature at four relative humidities, with the classification boundaries applied by the firmware.

3.9 Battery and Helmet Presence

Battery indication begins at the divider on GPIO 36, where an upper resistor of 100 kΩ and a lower one of 47 kΩ bring the pack voltage inside the converter range and give a reconstruction multiplier of 147/47. Main averages twenty millivolt readings every five seconds with 500 µs between samples, then multiplies the converter side voltage by that ratio to recover the pack voltage. The percentage is a rounded linear mapping between 6.40 V and 8.40 V, clamped at both ends. A value outside the range is clipped and never extrapolated. What the indication reflects is therefore the configured voltage window and nothing else, since the code holds no discharge curve and performs no charge integration, hence remaining operating hours cannot be inferred from it at all.

Presence uses GPIO 39, with the FSR402 documented towards 3.3 V and a 10 kΩ resistor towards ground, and ten readings averaged every 200 ms. While the helmet is unworn a value of at least 1,100 will change the state to worn, and once worn a value of at most 700 clears it. Between the two the preceding state simply remains. That gap gives hysteresis, so a wearer adjusting the strap does not produce a run of state changes, when a single threshold would have produced exactly that. The

counts themselves still need testing against the actual fit and mechanical loading. One more point matters here and is easy to miss, which is that presence does not switch the hazard routines off anywhere in the supplied loop.

Equation 3.6 recovers the pack voltage from the divider tap and Equation 3.7 converts it to a percentage. The second of these assumes the pack discharges linearly between the two configured voltages, and a lithium cell does not.

$$V_{pack} = V_{adc} \times (R_1 + R_2) / R_2 \tag{3.6}$$

$$\text{SoC} = 100 \times (V_{pack} - V_{empty}) / (V_{full} - V_{empty}) \tag{3.7}$$

3.10 Warning Arbitration

Output arbitration is called after sensor processing and it resolves a fixed order. Gas is considered first, then impact, then a valid elevated heat index, and only when all three branches are inactive can a BLE indication reach the shared outputs. Gas WARNING lights yellow with no sound. HIGH WARNING switches both LEDs every 300 ms, when DANGER holds red steady and switches the shared buzzer every 200 ms. Impact holds red and the buzzer on together. For the environmental temperature function, the calculated heat index is classified as Normal (HI 34 degrees Celsius or below), Caution (above 34 up to 39 degrees Celsius), Warning (above 39 up to 45 degrees Celsius), or Danger (above 45 degrees Celsius). The local temperature buzzer warning is activated only when the status reaches Danger. The temperature and impact functions share the single buzzer on GPIO 33. The temperature warning remains active while the status is Danger and is cleared when a valid later reading returns to Warning or a lower status, provided that no impact alert is active.

Every warning branch drives one buzzer, which is connected to GPIO 33 and driven active high. The buzzer sounds for a gas danger, for a confirmed impact alert, for a heat-index status of Danger and for a red geofence zone. For the temperature function, Caution and Warning light the indicators without sounding the buzzer.

Receiver commands occupy the lowest priority. A red command requests steady red with sound, a yellow command requests yellow with a 150 ms buzzer half cycle, and the check in confirmations and output tests run as timed patterns that expire on their own. Main stops applying any receiver command once its report is being ten seconds old, which prevents a stale instruction from holding an output indefinitely after the link drops. Telemetry carries both the report age and a freshness flag, hence allowing the computer to separate a current command from an expired one. The timeout limits how long a stale indication can persist. It does not prove that any particular command ever reached the helmet.

Figure 3.7 draws the same order as a decision path. Reading it from the top, the first branch that matches takes the outputs and every branch below it is skipped for that pass, hence a gas reading above the safe band will suppress a heat index warning entirely and the wearer sees only the more urgent of the two. That is deliberate. Two warnings competing for one buzzer produce a pattern that means neither of them.

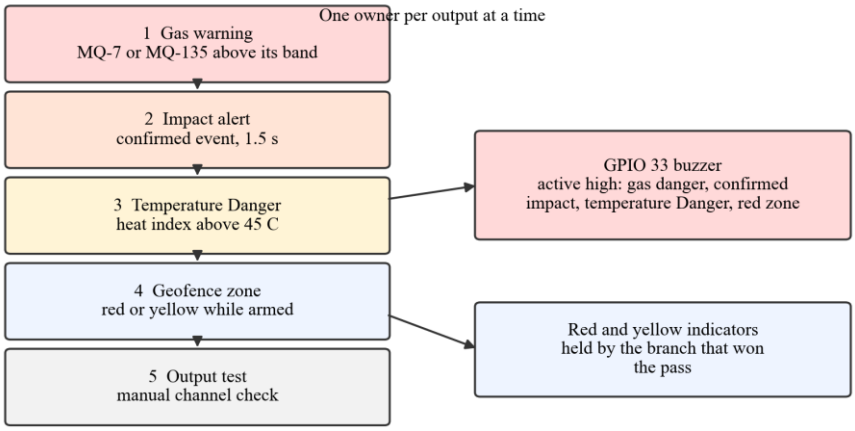


Figure 3.7: Order in which the branches are offered the shared outputs, with one owner per pass.

3.11 BLE Distance and Operating Modes

Receiver identifies the helmet using both its advertised name, TAG001, and the configured service UUID. An advertisement with a different name or service is excluded by the scan callback. The first accepted RSSI reading initialises the filter. Later readings update it according to Snew = 0.35r + 0.65Sprevious. This gives more weight to the retained value than to the new sample, reducing the immediate effect of a single fluctuation while also delaying the response to a sustained change.

Distance is estimated using d = d0 × 10^[(P0 − S)/(10n)]. The initial settings are a reference distance d0 of 1 m, measured power P0 of −59 dBm and path-loss exponent n of 2.7. The calibration controls can change these values or capture the filtered RSSI as measured power. Capturing a reading does not establish the physical reference distance. The operator must first place the helmet at the declared distance, and the calibration remains conditional on that arrangement.

The default red boundary is 3 m and the outer boundary 5 m. Red is entered below 3 m and retained below 3.4 m. The near region can remain active below 5.4 m when the preceding state was inside, giving the configured 0.4 m hysteresis. A missing advertisement for 2,500 ms clears detection and leads to OUTSIDE. That state may therefore mean loss of reception as well as movement beyond a boundary; the interface should be interpreted with the detection status in view.

The BLE worker scans for one second, evaluates state and attempts a characteristic write to Main, then disconnects and delays 200 ms. Connection time contributes to the cycle. Adding the scan and delay alone would not give a verified update period. Main reports the received BLE information through its own HTTP telemetry, which gives the host a second view of the command path alongside Receiver's direct status.

GEOFENCE, TAGGING and TEST are selectable modes with different purposes. Geofencing selects an indication from the zone, tagging applies attendance rules, and test mode requests individual output checks. Receiver simulation supplies a labelled distance to its state logic but suppresses physical BLE writes. Hence

simulation can exercise a displayed state transition without establishing that a real packet was received or a helmet output operated. A complete communication test must record the receiver decision, successful delivery and the resulting Main state as separate observations.

3.12 Distance Model and Zone Decision

The receiver converts a received signal strength indication into a distance using the log distance path loss relationship. Taking the measured power at the reference distance as A, the path loss exponent as n, the reference distance as d0 and the filtered indication as RSSI, the distance d is being obtained from the following equation.

The firmware implements exactly this expression, with A defaulting to -59 dBm, n to 2.7 and d0 to 1 m, and every one of the three is adjustable at runtime through the calibration endpoint. The exponent carries the environment. A value of 2 describes free space, when a value approaching 3 describes a room with furniture and walls that absorb and reflect, and the default of 2.7 is a compromise chosen for an indoor laboratory and not a measured figure for any particular site. Table 3.5 lists these parameters with their symbols, their defaults and the ranges the receiver will accept.

Raw indications are noisy enough that a distance computed from a single packet is not usable. An exponential moving average is applied on arrival, with the new sample weighted at 0.35 and the retained estimate at 0.65, so the filtered value settles over roughly the last few sightings. The first sighting after a quiet period initialises the filter to the raw value instead of averaging into a stale one. This is a deliberately cheap filter and the literature offers better; bagging over repeated samples improves the calibration of BLE distance estimates (Maratea, Salvi and Gaglione, 2019), and combining signal strength with inertial readings removes a large part of the unreliable detections (Filus et al., 2022). Neither is implemented here.

Two boundaries are then applied to the filtered distance, at a near radius of 3 m and an outer radius of 5 m by default. A single boundary would chatter, because a worker standing at the threshold produces a filtered distance that crosses back and forth, and each crossing would emit a command. The firmware adds a hysteresis margin of 0.4 m, applied only in the direction that keeps the wearer in the zone they already occupy. Hence causing a tag that has entered the red zone to remain in it until it retreats past the boundary plus the margin.

The result is a zone and not a position. One receiver produces one range estimate and a range estimate without bearing describes a sphere. The design accepts this and reports RED, YELLOW or OUTSIDE, which is the most that a single anchor can support honestly. A worker 3 m to the left and a worker 3 m behind will be reported identically.

Figure 3.8 plots the computed distance against filtered signal strength for three exponents, with the default boundaries marked. The curves diverge sharply at the weak end. Which is why an exponent fitted in the wrong room will misplace the outer boundary by several metres while the near boundary still looks reasonable.

Equation 3.8 is the log distance relationship the receiver evaluates on every sighting, when Equation 3.9 is the filter applied to the indication before that evaluation happens. The filter matters more than the model does, since a single unfiltered packet will move the computed distance by metres.

$$d = d_0 \times 10^{(A - \mathrm{RSSI}) / (10n)} \tag{3.8}$$

$$\mathrm{RSSI_f}[k] = 0.35 \times \mathrm{RSSI_{raw}}[k] + 0.65 \times \mathrm{RSSI_f}[k - 1] \tag{3.9}$$

Table 3.5: Distance Model Parameters and Their Permitted Ranges

Parameter	Symbol	Default	Accepted range
Measured power at the reference distance	A	-59 dBm	-120 to -10 dBm
Path loss exponent	n	2.7	0.5 to 8
Reference distance	d0	1 m	0.1 to 20 m
Near boundary	Not applicable	3 m	0.1 to 20 m

Parameter	Symbol	Default	Accepted range
Outer boundary	Not applicable	5 m	0.2 to 30 m
Hysteresis margin	Not applicable	0.4 m	0 to 5 m
Filter weight on the new sample	Not applicable	0.35	Fixed in firmware
Sighting timeout	Not applicable	2500 ms	Fixed in firmware

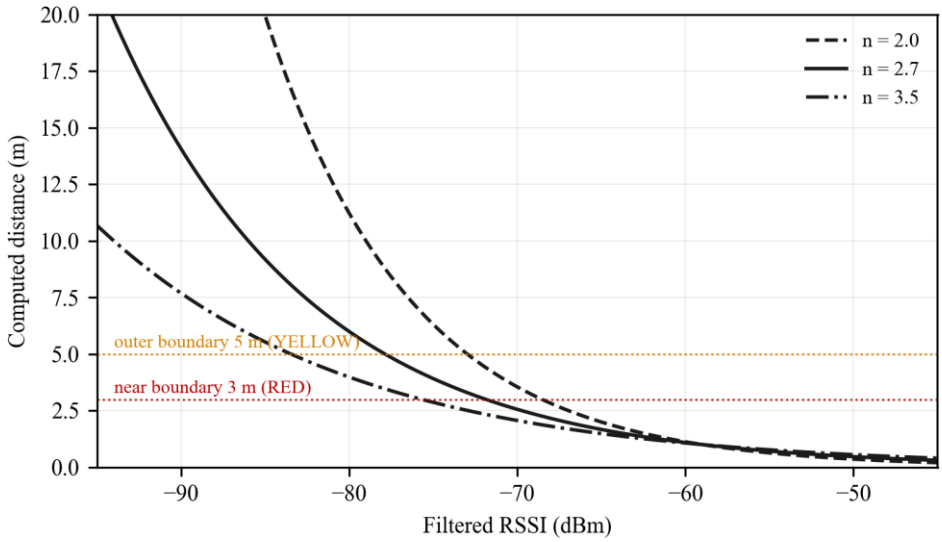


Figure 3.8: Computed distance against filtered RSSI for three path loss exponents, with the default zone boundaries.

3.13 Attendance and Receiver Time

Attendance uses the receiver clock together with the proximity state. Entering either near zone starts the configured dwell interval. If the entry remains valid for that interval, Receiver records check-in and compares the local minute with the expected check-in minute to determine lateness. The default dwell and re-entry cooldown are zero. Checkout follows five continuous seconds outside. These rules concern the presence of the configured tag; they do not independently establish who is wearing the helmet.

The initial schedule specifies 09:00 check-in, 21:00 checkout and sixty minutes of checkout grace. Grace belongs to the overdue deadline, not the late-arrival comparison. Receiver forms the deadline using the check-in day and adds a day when the configured checkout time precedes the configured check-in time. This permits an overnight schedule without treating the setting as a duration of several days. A changed schedule recalculates the current deadline, so its effect should be considered when demonstrating an already active visit.

The host sends an epoch and timezone offset to establish receiver time. Receiver then combines its stored offset with elapsed milliseconds for subsequent decisions. The manual time control allows the operator to demonstrate a later part of the attendance period without waiting until that real clock time. At the host, a manual override prevents ordinary polling from immediately replacing the chosen time with computer time. The existing Python test checks that host behaviour using a mocked receiver. It does not establish the receiver's clock drift or persistence across power loss.

Mode, calibration and roster settings are stored through ESP32 Preferences. Clock synchronisation is held in memory. The distinction matters after restart because saved operating settings do not imply that the receiver has a valid current time. Attendance records occupy a 64-entry memory ring, with a boot identifier and event sequence used by the computer to distinguish records. If collection stops for long enough to overwrite an entry, later polling cannot reconstruct that missing event from the ring.

The roster associates the single tag with a worker name, identifier, role and station. It supplies descriptive information to later records, rather than adding another BLE identity. The dashboard can change this configuration through the receiver API. Changes should be made deliberately between demonstrations, since a stored name and an observed tag are different kinds of evidence. The event register is useful for reviewing transitions, but the interpretation of those transitions remains conditional on receiver time, reception and the selected roster.

The receiver settings in Table 3.6 establish the initial proximity and attendance configuration. Changing these values changes the interpretation of the same reception or clock input. Table 3.7 lists the principal HTTP interfaces used to observe and control the devices. The host provides a common interface, but the firmware routes remain separate endpoints on the network. This separation is relevant when diagnosing whether a failure concerns the computer connection, receiver decision or helmet response.

Figure 3.9 shows the states the receiver holds and what moves between them. The dwell and the cooldown timers are the two controls that keep a person walking past the door from generating a check-in and an immediate check-out.

Table 3.6: Initial Receiver Configuration

Parameter	Initial value	Interpretation
Tag	TAG001 + service UUID	Both filters required
RSSI filter	0.35 new + 0.65 prior	First reading initialises filter
Reference model	1 m; −59 dBm; n = 2.7	Configurable model, not field calibration
Zone boundaries	3 m / 5 m	0.4 m hysteresis
Lost detection	2,500 ms	No accepted advertisement
Entry dwell / cooldown	0 s / 0 s	Configurable
Outside checkout	5 s	Continuous outside state
Expected attendance	09:00–21:00	Hours and minutes
Overdue grace	60 min	Added to checkout deadline
Event ring	64 entries	Volatile receiver record buffer

Table 3.7: Principal HTTP Interfaces

Owner	Method and route	Purpose
Main	GET /api/telemetry	Sensors, alerts and received BLE state
Receiver	GET /api/status	Zones, attendance, events and settings
Receiver	POST /api/mode	GEOFENCE, TAGGING or TEST
Receiver	POST /api/settings	Attendance hours and delays

Owner	Method and route	Purpose
Receiver	POST /api/time	Epoch and timezone offset
Receiver	POST /api/calibrate	Distance-model / zone settings
Receiver	POST /api/roster	One-tag worker information
Receiver	POST /api/simulate	Labelled distance input
Receiver	POST /api/selftest	Output-test command
Host	GET /api/state	Combined state and local records
Host	GET /api/cloud/state	Cloud current values / history
Host	GET /api/export/csv or xlsx	Event-register export
Cam	GET /status, /capture, /control	Camera state, JPEG and settings
Cam	GET /stream (port 81)	Image stream

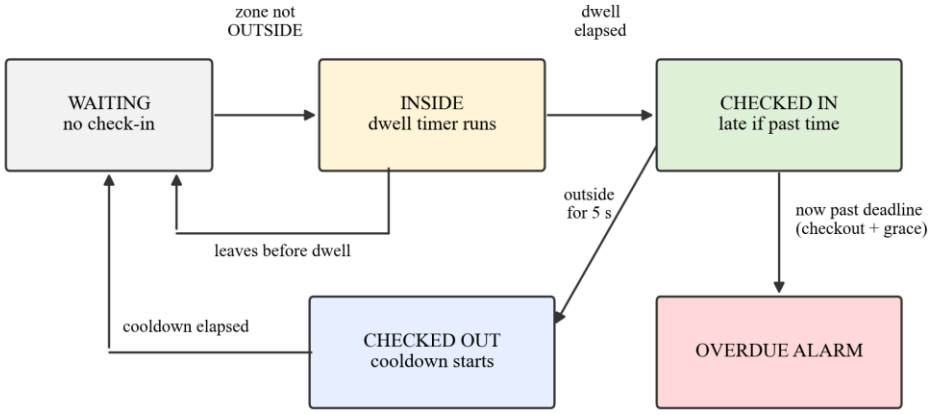


Figure 3.9: Attendance states held by the receiver and the conditions that move between them.

3.14 Camera and Local Records

Cam provides a separate camera service using the AI Thinker pin configuration. Setup requests JPEG capture with a 20 MHz external camera clock. With PSRAM available, the program allocates two frame buffers and requests the latest frame; the alternative path uses an SVGA allocation in DRAM. After initialisation, the operating JPEG frame size is set to QVGA. These are allocation and configuration choices visible in

the sketch. They do not establish the frame rate or image delay obtained on a particular board.

The server exposes status, still capture and sensor controls alongside streaming. The desktop host proxies a still capture as JPEG and permits selected camera settings, including image adjustment, orientation and flash intensity. There is no image-based hazard classifier in this path. An operator can observe the camera output, but the existence of a stream should not be described as automatic recognition of a vehicle or unsafe act.

The host polls Main and Receiver concurrently through HTTP. Their functions remain distinct: Main supplies the sensor measurements and forwarded BLE state, while Receiver supplies its own proximity, attendance and configuration state. Main telemetry is sampled into SQLite at ten-second intervals. On a failed request, the host retains prior values and marks the affected connection offline with an error. A retained value therefore needs its connectivity context; it cannot automatically be interpreted as a fresh measurement.

Receiver events are imported using boot and sequence identifiers. Main impact records use Main's boot identifier and impact sequence. SQLite applies a unique event key so that replaying a previously imported record does not create a duplicate. The database is the local record used to regenerate CSV and Excel exports. If a spreadsheet application prevents replacement of an export file, the event can still remain in SQLite and the export can be retried after the lock is released.

Formula-like text is prefixed before export to avoid treating selected cell contents as formulas. Export files are replaced through temporary files, separately from the dashboard's current filter. These measures address specific data-handling cases; they do not amount to a general guarantee against storage failure. The supplied tests use temporary data directories and mocked errors to check these paths. Their successful assertions can support the implementation discussion without being presented as a power-loss test or a measurement of field reliability.

3.15 Camera Subsystem Implementation

The camera is the most self contained of the four programs and also the one whose limitations are most often misread, hence this section describes what it does and then what it does not do. It runs the Espressif camera web server on an AI Thinker ESP32-CAM board, with the sensor clock at 20 MHz and JPEG as the pixel format.

Frame buffer allocation depends on what the board actually has. Where PSRAM is present the program keeps two buffers in that external memory at JPEG quality 10 and grabs the latest frame, when a board without PSRAM falls back to a single buffer in internal DRAM and the frame size is reduced to SVGA to fit. After initialisation the running frame size is dropped to QVGA regardless, which trades resolution for a frame rate the radio can sustain. A viewer therefore receives a smaller image than the sensor can produce, and raising it through the control endpoint will lower the rate.

The server exposes the standard set of endpoints. A single still is available at capture, a multipart stream at stream, an uncompressed bitmap at bmp, the sensor settings at status and a setter at control, alongside register level access at reg, greg, pll, xclk and resolution. The host dashboard uses only three of these and it filters the control endpoint against a fixed list of permitted setting names, with the value range checked before the request is forwarded.

That filtering on the host is worth noting because the camera itself performs none. The register level endpoints accept arbitrary addresses and values, and nothing on the camera requires authentication of any kind. Any device on the same network can reach them. The dashboard's allow list protects the dashboard's own users from a malformed request; it does not protect the camera, since the camera is reachable directly. This is being recorded as a limitation and not as a control.

There is no recording path. Frames are served on request and are never written to storage, hence an event captured by the sensors has no image associated with it anywhere in the record. Adding one would mean capturing a still when an alert is raised and storing it beside the event row, which is straightforward and is listed in Section 5.2. As built, the camera is a live view only.

During operation, the ESP32-CAM first initialises its onboard camera in JPEG mode and then connects to the configured Wi-Fi network in station mode. For the current prototype test network, a static local IP address of 10.249.219.150 is configured with gateway 10.249.219.22 and subnet mask 255.255.255.0 before the Wi-Fi connection is started. Wi-Fi sleep is disabled in the program to support more stable streaming. When the connection is successful, the ESP32-CAM starts the CameraWebServer and the camera interface can be opened through the local IP address by a device on the same network. The initial operating resolution is set to QVGA (320 x 240), although the resolution can later be adjusted through the camera web interface. When PSRAM is available, the program uses two frame buffers, JPEG quality setting 10 and latest-frame grabbing to improve the streaming operation. QVGA resolution, JPEG settings, PSRAM frame buffering, latest-frame grabbing and disabled Wi-Fi sleep are used to support stable streaming and reduce delay. Table 3.8 lists the camera endpoints together with the use the host makes of each one.

Table 3.8: Camera Endpoints and Their Use by the Host

Endpoint	Purpose	Used by the dashboard
/capture	Single JPEG still	Yes, proxied by the host
/stream	Multipart JPEG stream	Yes, opened directly
/status	Current sensor settings as JSON	Yes
/control	Set one sensor variable	Yes, against a fixed allow list
/bmp	Uncompressed bitmap frame	No
/reg and /greg	Direct sensor register write and read	No
/pll and /xclk	Clock configuration	No
/resolution	Sensor window configuration	No

3.16 Host Application Architecture

The host application is the only part of the system that holds state across a restart, hence its structure decides whether the record can be trusted. It is a Flask application served by Waitress, with two background threads and a small thread pool, and a single SQLite file underneath.

Acquisition runs on its own thread. Each cycle submits two fetches into the pool, one to the Main telemetry endpoint and one to the receiver status endpoint, so that a slow device does not delay the other. Both use a connect timeout of two seconds and a read timeout of four, which bounds the cycle. A failure is caught per device and not per cycle. The link is marked offline and the error string is stored, when the last known values for that device are left untouched, so a dashboard showing a stale battery percentage also shows that the link is down instead of showing a zero.

The second thread handles the cloud. It writes the receiver branch and then walks the unsynchronised events in order, marking each one as it succeeds, and it holds no lock while waiting on the network. Simulated events are marked synchronised without being uploaded, which keeps test traffic out of the cloud record entirely. Any error message is scrubbed of the Firebase token before it is stored, in both plain and percent encoded form, since a requests exception will otherwise carry the full query string including the credential.

Clock handling deserves its own note because it is the one place where the host overrides a device. The receiver has no real time source. On each poll the host checks whether the receiver reports itself synchronised, and if not it pushes the current epoch with the local offset. An operator who has set the clock manually would find that setting overwritten on the next cycle, hence a manual override flag suppresses the automatic push until the application restarts. This behaviour is being exercised directly by the test suite.

Three protections sit on the web layer. The maximum request body is capped at 64 kB, any POST carrying an Origin header from another host is refused with 403, and device addresses submitted through the connections endpoint are parsed and rejected unless they are plain HTTP or HTTPS without embedded credentials. None of this is authentication. The dashboard has no login of any kind and anyone who can reach the port can drive the receiver, which is acceptable only because the design assumes a single trusted machine on a private network.

The origin check is not decoration. A cross site request forgery works precisely because a browser will attach a user's ambient authority to a request that a different site caused it to make, and the attack has been documented in the security literature for many years (Shema, 2010). Here the dashboard has no login at all, hence there is no session to ride, and the check is being applied as a second line so that a page open in another tab cannot drive the receiver on the operator's behalf. Table 3.9 lists the endpoints the host exposes to the browser.

Table 3.9: Host Application Endpoints

Route	Method	Function
/api/state	GET	Full snapshot with history and events, cloud token excluded
/api/cloud/state	GET	Firebase current values and one history branch
/api/receiver/(endpoint)	POST	Forward a whitelisted command to the receiver
/api/clock	POST	Set the receiver clock and optionally lock it
/api/connections	POST	Persist validated device addresses
/api/export/(csv or xlsx)	GET	Download the complete attendance register
/api/exports/flush	POST	Rewrite the on-disk export files
/api/camera/status, control, capture	GET and POST	Filtered camera access

3.17 Cloud Interfaces and Access Controls

The cloud paths are split between Main and the computer host. Main retains direct uploads of current sensor values, selected histories and impact records. A separate worker in the host sends receiver state and locally recorded events to Firebase. The local database remains useful if cloud communication fails, because an event need not disappear merely because its upload has not succeeded. This arrangement also means that an operator must identify which component is responsible when a cloud branch stops updating.

The host uses deterministic event paths and marks events as synchronised after successful requests. Repeating a write to a known path is different from generating a new child on every retry. Firebase documents PUT as replacement at a specified path, PATCH as an update of selected children and POST as creation of a new child key (Google, 2026). Those semantics explain why request method and path matter when assessing duplicate records. Their use in source code does not establish that a configured database accepted the request. A live test needs a response and a read-back of the expected record.

The prototype has several bounded controls. Receiver commands pass through an allowlist at the host, numeric inputs have range checks, and browser control requests with a different Origin are rejected. The dashboard state does not expose the configured Firebase token. These measures address selected application paths. They do not provide a login system for the device endpoints, and an Origin check alone does not authenticate an operator.

The inspected ESP32 HTTP routes have no enforced login, and the BLE command interface has no configured pairing security. Main also calls setInsecure() on its Firebase client, explicitly bypassing certificate verification. The report treats this as a deployment limitation. A shared wireless network provides connectivity; joining it does not by itself establish the trustworthiness of every client capable of reaching a controller.

A deployment check must therefore consider the direct device paths as well as the dashboard. Restricting a button in the interface cannot prevent a request made directly to an unprotected receiver endpoint. Certificate verification, authenticated control and suitable network access restrictions are required follow-up work. They should be tested with both permitted and rejected requests, retaining the response and resulting device state. This is a verification recommendation derived from the current access paths, rather than a claim that an attack was performed during the project.

3.18 Power Budget and Battery Estimation

The helmet is battery powered, hence the current drawn by each part decides how long a shift the prototype can cover, and the figures below are taken from the manufacturers' published specifications rather than from measurement. They are being presented as a design budget and not as a result. Table 3.10 collects them.

The dominant consumer is not any sensor. It is the radio. An ESP32 transmitting over Wi-Fi draws on the order of a few hundred milliamps in bursts, and the MQ series heater is the only element in the same class, since a tin dioxide sensor holds its element hot continuously and cannot be duty cycled without losing its response. Two MQ sensors running together therefore set the floor of the budget before any controller is switched on. The remaining parts, the light sensor, the inertial unit, the temperature sensor and the divider network, are small enough that they will not change the conclusion.

The battery is a two cell lithium ion pack, treated by the firmware as full at 8.40 V and empty at 6.40 V. State of charge is being estimated by linear interpolation between those two voltages after a resistor divider of 100 kΩ over 47 kΩ brings the pack voltage inside the ADC range, with twenty samples averaged per reading. A linear map of terminal voltage is the crudest of the accepted estimators. Under load the terminal voltage sags, hence the reported percentage will read low while the lamp and

both heaters are active and will recover when they stop, and the same pack can report two different percentages within a minute without anything being wrong with the pack.

The divider itself has an implication that is easy to overlook. With 100 kΩ and 47 kΩ across a fully charged pack, a continuous quiescent current flows through the divider for as long as the battery is connected, which will discharge the pack slowly even when the controller is asleep or unpowered. A prototype left connected on a bench over a weekend can be found flat for this reason alone. The correct fix is a switched divider or much larger resistances, and neither is in the present build.

No measured runtime is reported in Chapter 4. Obtaining one requires an instrumented discharge on the actual assembled helmet, which is listed in Section 4.5 as pending work with the table prepared for the figures.

Table 3.10: Indicative Power Budget from Published Device Specifications

Element	Typical current	Duty	Note
ESP32 with Wi-Fi active	95 to 240 mA	Bursty	Transmit peaks dominate the budget
ESP32 with BLE only	30 to 90 mA	Continuous	Receiver scans one second in five
MQ-7 heater	About 150 mA	Continuous	Cannot be duty cycled without losing response
MQ-135 heater	About 150 mA	Continuous	As above
Helmet lamp at maximum command	Load dependent	Ambient driven	Command capped at 178 of 255
BH1750, MPU6050 and DHT22	Under 5 mA combined	Continuous	Negligible against the heaters
Battery divider, 100k over 47k	About 57 uA at 8.4 V	Continuous	Flows whenever the pack is connected
ESP32-CAM while streaming	180 to 310 mA	While viewed	Separate supply in the prototype

3.19 Proposed Physical Evaluation

Sensor evaluation should begin with reference measurements. The light input can be compared with a reference lux meter at several illumination levels while the commanded PWM value is recorded. Temperature and humidity can be compared with a reference instrument after readings settle. Battery voltage should be measured at the supply and divider output before comparing the displayed estimate. A discharge experiment is needed to relate that estimate to operating time. For the gas channels, a controlled calibration arrangement and an appropriate reference are required before any concentration claim; the present normalised ADC indices cannot supply that evidence.

Motion evaluation requires an instrumented fixture and defined movements. Ordinary handling, walking and controlled mechanical disturbances should be recorded separately from candidate impacts, with timestamps for acceleration, rotation and output activation. A labelled set would allow false detections and missed detections to be counted. No person should be subjected to an impact for this purpose. Boundary cases around the acceleration threshold, rotation threshold and confirmation timeout are especially useful because the source checks rotation before discarding an expired candidate.

BLE evaluation should hold known separation distances while varying helmet orientation and obstruction in the intended room. Each trial should retain raw RSSI, filtered RSSI, estimated distance, zone and detection status. Repeated approaches and departures would show whether hysteresis prevents rapid zone changes near a boundary. The attendance procedure should then exercise entry dwell, exit delay, repeated visits, cooldown, overdue activation and a schedule crossing midnight. Manual clock changes and receiver restarts should be included. These records would distinguish a timing-rule defect from a fluctuating radio measurement.

Combined-output testing should apply overlapping requests and record the actual LED and buzzer states. Gas, impact, temperature and BLE conditions must be considered together because a single-module test cannot establish priority behaviour.

Communication interruptions should also be introduced while counting generated and received events. The resulting record can reveal whether the bounded receiver queue or latest-impact slot loses intermediate information. Camera frame rate, capture success and cloud recovery require their own observations. Table 4.2 relates these outstanding checks to the objectives; no proposed trial in that table is reported as an achieved result.

3.20 As-Built Hardware

The controllers and the sensors are being mounted into a standard industrial helmet shell, and the photographs in this section record the state of that build at the time of writing. Figure 3.10 shows the interior, where the ESP32 DevKit board sits on a strip board together with the buck converter and the shared buzzer, when the two gas modules are wired out towards the front of the shell. The 2S pack is held at the rear so that the weight is not being carried at the front of the head. Wiring is hand soldered and it is routed through the shell webbing, hence the untidy appearance of the harness in the photograph.

Sensor placement follows what each device has to observe. The two MQ elements are mounted through the front of the shell as shown in Figure 3.11, so that the sensing face will meet the outside air instead of the enclosed volume under the helmet. Figure 3.12 gives the side view with the red and the yellow indicator at the top edge of the shell, which places them inside the peripheral vision of a worker standing nearby but not inside the vision of the wearer. Figure 3.13 shows the BH1750 and the DHT22 at the crown, where the light sensor faces upward for the ambient reading when the temperature sensor is kept away from the regulator so that the heat of the electronics will not be measured as the heat of the workplace. The FSR402 sits under the harness and it is not visible in any of the four photographs.

Two limitations follow from this arrangement and both of them are being carried into Section 4.5. The gas elements are exposed at the front, hence a headwind

will reach them before it reaches the face of the wearer, when a still condition will leave the same elements reading an almost stagnant sample and the reported percentage will drift with it. No enclosure is fitted.

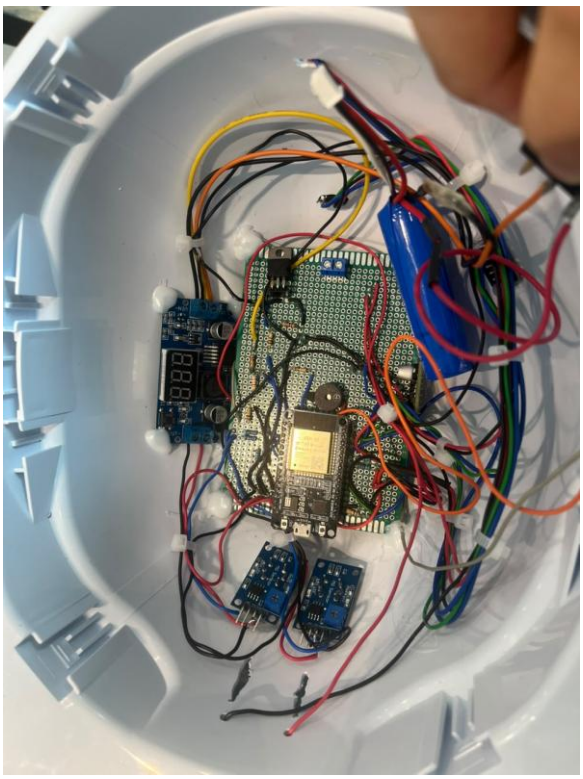


Figure 3.10: Interior of the shell, showing the controller board, the converter and the harness as they are being mounted.



Figure 3.11: Front of the helmet with the two gas elements mounted through the shell and the lamp below them.



Figure 3.12: Side view of the assembled helmet, where the indicators sit at the top edge of the shell.



Figure 3.13: Crown of the helmet carrying the BH1750 and the DHT22, with the two indicators above them.

3.21 Laptop Attendance, Voice Assistant and Emergency Reporting

The attendance decision has been moved off the second controller and onto the computer. The helmet advertises as TAG001 and the computer measures that advertising with its own Bluetooth adapter, hence the boundary is a radius around the computer and not around a separate receiver. Two switches arm the function separately. A check-in is recorded after the helmet stays inside a one metre radius for one second while the check-in switch is on, when a check-out is recorded after the same helmet stays outside that radius, or out of radio range altogether, for one second while its own switch is on. Each of them happens once and a reset button starts the next day. The second controller keeps a simpler task in this arrangement: it advertises as a fixed beacon and the helmet scans for it, so the geofence zone is being decided on the helmet itself.

Speech is handled by two cloud calls and neither of them is being given any control of the hardware. The operator holds a button, the browser records, and the recording goes to a speech model which returns the words. Those words are passed to a language model together with a snapshot of the dashboard, which carries the readings, the zone, the attendance state and the last ten records. The model answers from that snapshot only and the answer is read back aloud. No control endpoint is reachable from the assistant, hence a wrong answer will never silence a buzzer or disarm the geofence.

Emergency reporting sits inside the same speech path and it is deliberately not left to the model alone. A fixed word list is matched against the transcript first, so a shout for help will raise the alarm without waiting for any network reply, when the model is asked separately to judge whether the speaker is reporting distress. Either route raises it. The screen is then covered by a red warning carrying the words that were spoken, a siren is played from the computer, and the record is written into the register with the trigger that caused it, where the warning stays on the screen until an operator acknowledges it. The helmet is not commanded at all.

Two limits are worth stating. A spoken report depends on a microphone at the computer and not on the helmet, hence a worker out of earshot of that machine cannot raise the alarm this way. The word list is English.

CHAPTER 4

RESULTS AND DISCUSSIONS

4.1 Execution Conditions

Software verification was carried out on the Python application supplied in Main.zip. The extracted application was tested without changes to its source files. On 16 September 2026, the command python -m unittest discover -s tests -v completed twenty-eight test methods in 2.070 s and returned exit code zero. The execution log is retained with the report. This duration describes the test suite on the computer; it does not represent the response time of the helmet.

The first execution did not reach the application assertions because the interpreter could not import requests. The supplied virtual environment contained the required dependencies, so its site-packages directory was added to the process-local Python search path for the second execution. No packages were installed and no application code was changed between these attempts. Recording the failed import is useful because the archive cannot be assumed to run under an arbitrary Python interpreter without its dependencies.

4.2 Telemetry and Record Handling

The tests used temporary storage directories, Flask test clients and substituted device responses. Therefore, the results assess selected host behaviours under controlled inputs. The test for Main telemetry supplied a payload containing BLE attendance information and an impact sequence. The attendance field appeared in the local API response and history, while the impact sequence produced an event record. This supports the requirement for the computer to consume BLE information forwarded through Main. The test did not transmit a BLE packet or apply an impact to a helmet.

Event storage was checked by submitting the same boot identifier and event identifier twice. The duplicate was rejected, and reopening the storage object retained one record. An event with a different boot identifier was then accepted. This result supports replay handling across device sessions within the tested storage path. Reopening an object is less demanding than an interrupted write or a computer power failure, so crash recovery remains outside this result.

The export checks covered both record formatting and recovery from a failed file replacement. A name beginning with =1+1 was exported as text, with a protective prefix, in the checked Excel cell and CSV output. A separate test substituted a permission error during Excel-file replacement. A later export succeeded and the original event remained in SQLite. These outcomes show why the database is retained as the primary record store. The permission error was simulated; the test did not open a workbook in Excel or interrupt the operating system. The assertions establish the observed behaviour of these examples without establishing protection against every spreadsheet formula or filesystem failure.

4.3 Timing and Interface Checks

Attendance control was examined at the host interface. The settings test returned HTTP 200 and forwarded the check-in, checkout, grace, dwell and cooldown values to

a substituted receiver-command function. Another test applied a manual clock change and then supplied a receiver response indicating that its clock was unsynchronised. The polling routine did not issue an automatic clock update. Together, these checks support preservation of the operator's timing settings within the Python control path. Whether the receiver applies those values correctly during an overnight attendance session was not exercised.

The failure-handling test supplied device timeouts. Main was marked offline while its previous battery value was retained and an error was recorded. The retained value of 70 was a test fixture. It is not a measured battery capacity or evidence of operating duration. Keeping the last value can help an operator understand the last known state, but the offline indication must be considered with it. A retained reading must not be interpreted as a current observation after communications have failed.

The remaining interface tests checked individual safeguards. A configured cloud token did not appear in the state response, a control request with an unrelated browser Origin was rejected with HTTP 403, and an epoch value of zero was rejected with HTTP 400. A changed Main address was written to the configuration file, while a file-scheme address was rejected. The demonstration test completed its polling path without the substituted device GET method being called and rejected a receiver-mode command. These are specific assertions. They do not constitute authentication testing, a complete credential audit or proof that every possible network operation is disabled in demonstration mode.

4.4 Test Summary and Objective Assessment

Table 4.1 summarises the twenty-eight executed test methods and their direct assertions. Every entry belongs to the recorded Python run; none represents a physical trial. Table 4.2 then maps the three project objectives to the available evidence and the outstanding measurements. The distinction prevents an implemented feature from being counted as a verified field result.

Figure 4.1 counts the evidence ledger by class and it is the most uncomfortable figure in the report. Twenty one entries were read in the source. One was exercised by an automated test. That ratio is the honest shape of the evidence behind this work, hence a reader should treat the implementation claims as verified by inspection and treat almost nothing as verified by execution.

Table 4.1: Executed Python Checks

Test focus	Observed assertion	Boundary
Laptop switches at start	Both attendance switches read off and no record is written	Logic only; no radio
Check-in dwell	Check-in raised after one second inside the radius, not before	Simulated RSSI samples
Switching on while inside	The one-second timer restarts when the switch is turned on	Logic only
Brief exit	A short step outside restarts the entry timer	Logic only
Check-out dwell	Check-out raised after one second outside the radius	Simulated RSSI samples
Lost signal	A helmet out of Bluetooth range counts as outside	No live adapter
Check-out preconditions	No check-out without its own switch and an earlier check-in	Logic only
One visit per day	No second check-in until the reset button is pressed	Logic only
Restart behaviour	The day survives a restart; the switches do not	SQLite reopened in place
Laptop distance model	Out-of-range values rejected; captured reference stored	No live adapter
Main offline	Link marked offline and the last values retained	Substituted device failure
Address discovery	Main found and its address remembered without typing one	Substituted responses
Geofence and impact records	Zone and impact events stored once each on repeat polls	Replayed payloads
Geofence warning	Warning on entry and on escalation, not on a missed report	Replayed zones
Alert signalling	Gas, impact and heat each signal once when they start	Replayed payloads

Test focus	Observed assertion	Boundary
Cloud documents	Current values and the history branches carry the expected fields	No network write
Command forwarding	Permitted Main commands forwarded; unknown ones refused	Mocked command
Attendance endpoints	Switch and reset endpoints change state as posted	Test client
Secrets	Neither cloud token nor assistant key appears in the state response	Test client
Cross-origin control	A cross-site control request is refused	Test client
Preview isolation	Preview mode performs no device traffic and refuses control	Synthetic data
Connection settings	Saved addresses persist and an invalid scheme is rejected	Temporary configuration
Assistant grounding	The snapshot sent carries the readings and no secret	Mocked model reply
Assistant authority	An assistant call leaves every helmet setting unchanged	Mocked model reply
Assistant without a key	A missing key and an empty question are both refused	Test client
Spoken distress	The keyword path raises the warning even when the model says otherwise	Mocked model reply
Ordinary question	A question about readings raises no warning	Mocked model reply
Voice route	Audio is transcribed first, then answered; an empty upload is refused	Mocked transcription

Table 4.2: Objective Assessment and Outstanding Evidence

Objective	Available support	Still required
1. Local monitoring	Six module routines and arbitration in Main source	Sensor references, labelled impacts, actual output timing
2. BLE and attendance	Receiver state logic; host settings and manual-clock tests	Radio trials, overnight sessions, reboot/time checks

Objective	Available support	Still required
3. Operator interface and records	Host telemetry/storage/export assertions; camera/cloud source paths	Live camera and cloud trials; sustained operation and event-loss counts

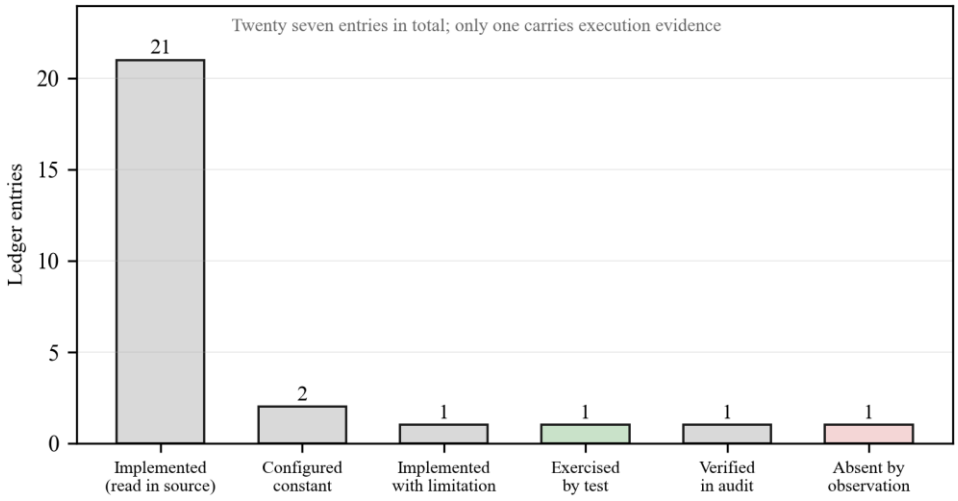


Figure 4.1: Evidence ledger entries counted by class of evidence.

4.5 Hardware Verification and Outstanding Measurements

This section is reserved for the physical results and it is deliberately left unfilled. Every measurement below requires the assembled helmet, and no such measurement has been taken at the time of writing, hence the tables are prepared with their columns and their procedures fixed so that the figures can be entered without changing the analysis around them. Nothing in this section should be read as a result. The highlighted entries mark what is outstanding.

The reason for separating this section from the rest of Chapter 4 is that the preceding sections rest on a different class of evidence. Sections 4.1 to 4.4 describe behaviour that was executed, in the sense that the Python suite ran against the real modules and the outcome was recorded. What follows here cannot be executed without hardware that is being built, and merging the two would let a reader carry the confidence of the earlier evidence onto claims that have none.

Five measurement groups are outstanding and each has a fixed procedure. The first is the light response, where the lamp command is to be recorded against a reference lux meter across the range from a dark room to full office lighting, since the firmware maps 20 lux and below to the maximum command of 178 and 150 lux and above to zero. What must be checked is whether the inverse quadratic curve between those endpoints produces a lamp that a wearer finds usable, and not merely whether the arithmetic is correct.

The second is the gas response. Two MQ elements are to be exposed to a known source and the raw counts logged, so that the baseline and raw maximum constants can be replaced with values taken from the actual elements. Until that is done the percentages in Section 3.6 remain a position between two assumed endpoints.

The third is the impact chain, which is the one requiring the most care. A helmet dropped onto a bench is not a head impact and the report will not present it as one. What can be verified without an instrumented head form is narrower, being whether a resultant acceleration above 2.5 g followed by a resultant rotation above 250 degree per second inside the 300 ms window will raise the alert, whether the 1500 ms alert duration and the 1000 ms cooldown behave as written, and whether the event reaches the host with its sequence advanced.

The fourth is the radio. The distance model needs its measured power and its path loss exponent fitted from readings taken at known separations in the actual room, and the zone boundaries then need checking for chatter while a person walks slowly across each boundary. The fifth is the battery, which needs an instrumented discharge from a full pack to the 6.40 V floor with the lamp and both heaters active.

The record sheets for these measurements follow the same order as the checks above, covering the ambient light response, the gas channel counts, the impact chain and the battery discharge. Each of them stays empty until the measurement is being taken, and the procedures that fill them are set out in Appendix E.

The helmet's own outputs were photographed while the prototype was running. Figure 4.2 shows the red indicator raised in a dangerous state, Figure 4.3 the yellow indicator in a warning state and Figure 4.4 both indicators together in a critical state. Figure 4.5 and Figure 4.6 show the ambient light reading as the surrounding illumination changes. These photographs record the implemented behaviour and are not measurements, so the calibration listed above is still outstanding.



Figure 4.2: Red LED lights up during dangerous situations.



Figure 4.3: Yellow LED turns on when at warning situations.

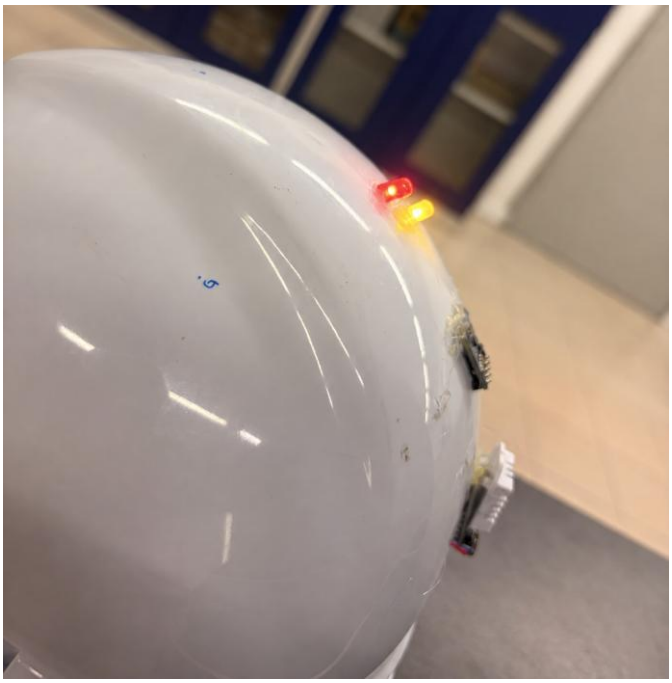


Figure 4.4: Both LEDs turn on when at critical situations.

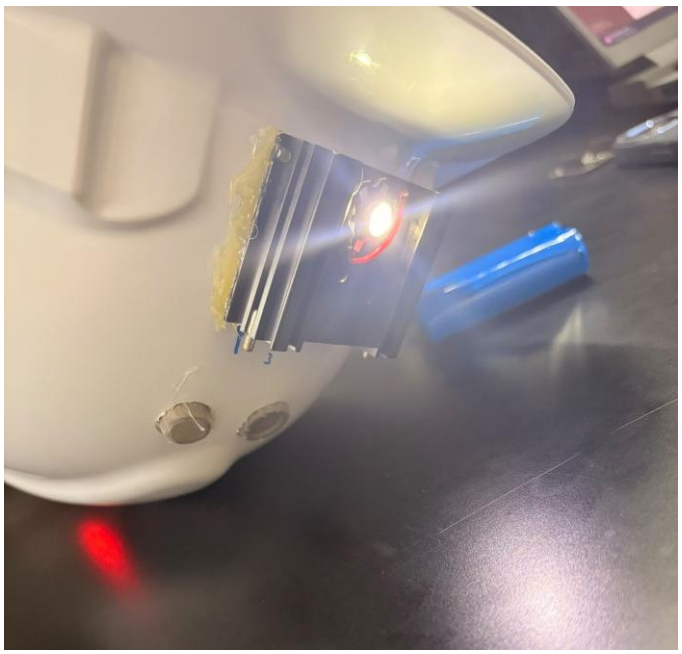


Figure 4.5: Ambient light shows darker when the surrounding environment is bright.

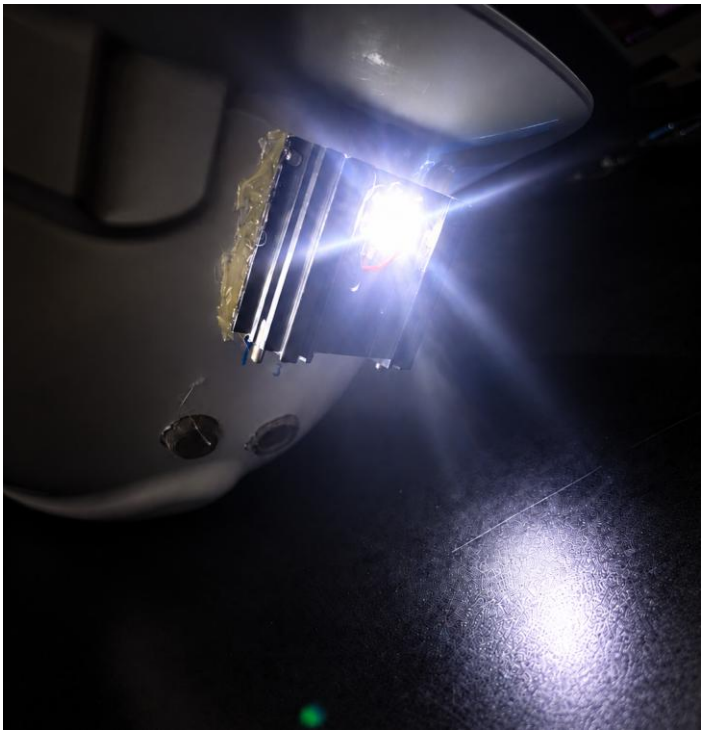


Figure 4.6: Ambient light shows brighter when the surrounding environment is dark.

The host application is being recorded in the same session, so that what the operator sees is on the record together with the tables above. Figure 4.7 shows the overview page with its fourteen indicator cards, and the values in it are live readings

taken from the assembled helmet and not calibrated measurements, hence the figure establishes that the acquisition chain is complete and nothing further than that. Figure 4.8 shows the attendance panel with both switches off, which is the state the host will hold at every start. Geofencing is shown in Figure 4.9 with the helmet inside the red boundary of the beacon, when Figure 4.10 carries the camera panel and the stored light history of the same session. Figure 4.11 shows the event register and the connection settings, and the rows in that register are zone entries and one check-in that were produced while the photographs above were being taken, hence their timestamps will not tally with any measurement table in this section.

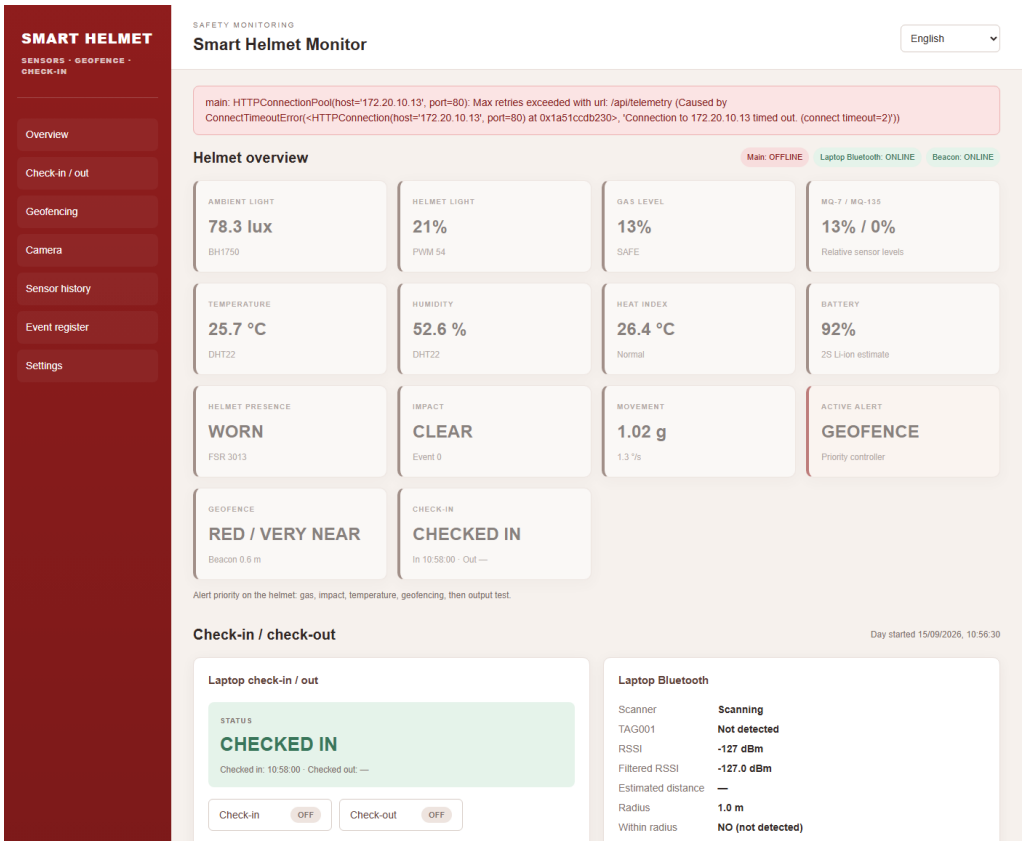


Figure 4.7: Overview page of the host application, carrying the fourteen indicator cards at the time of the session.

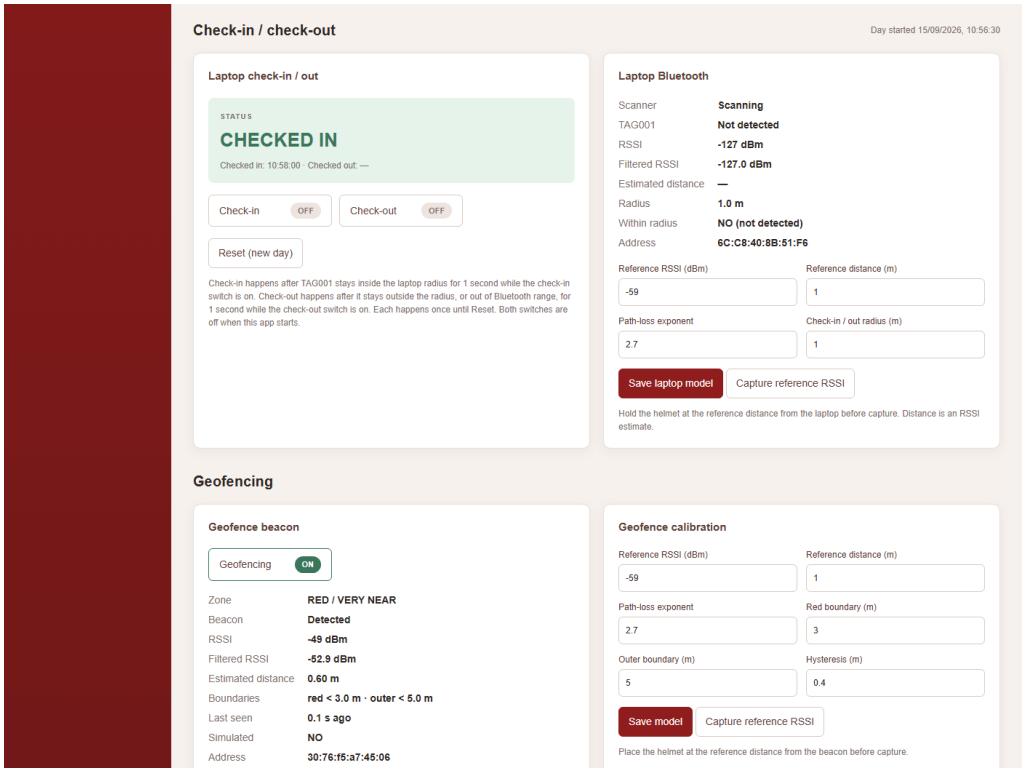


Figure 4.8: Check-in and check-out panel, with both switches in the off state the host will hold at every start.

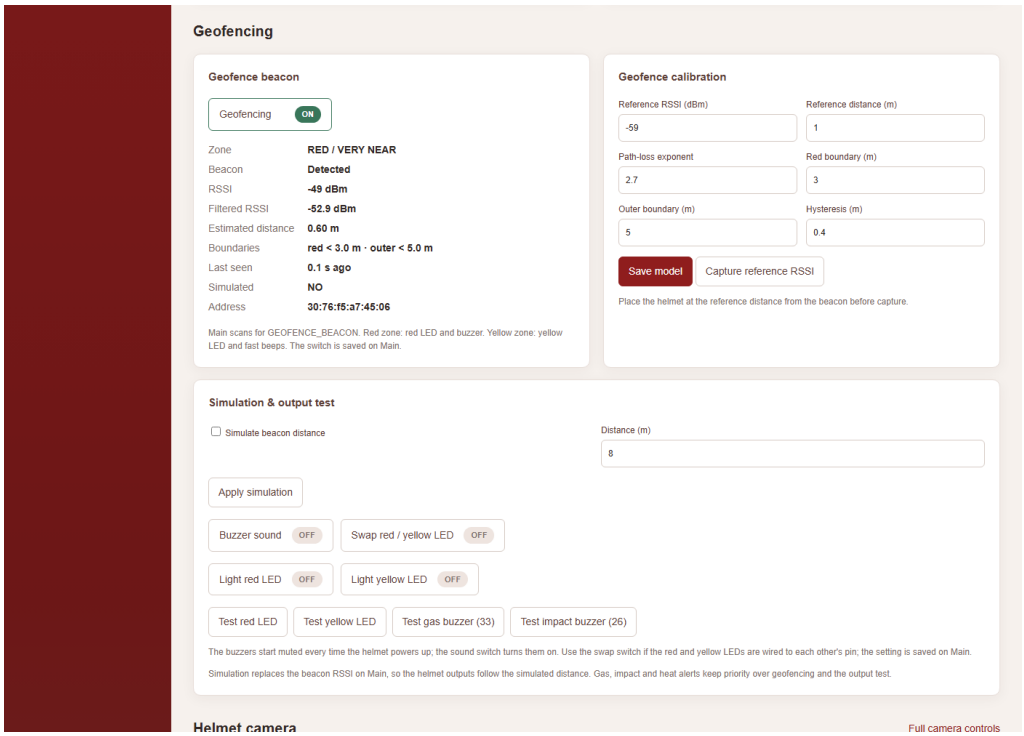


Figure 4.9: Geofencing panel with the helmet inside the red boundary of the beacon.

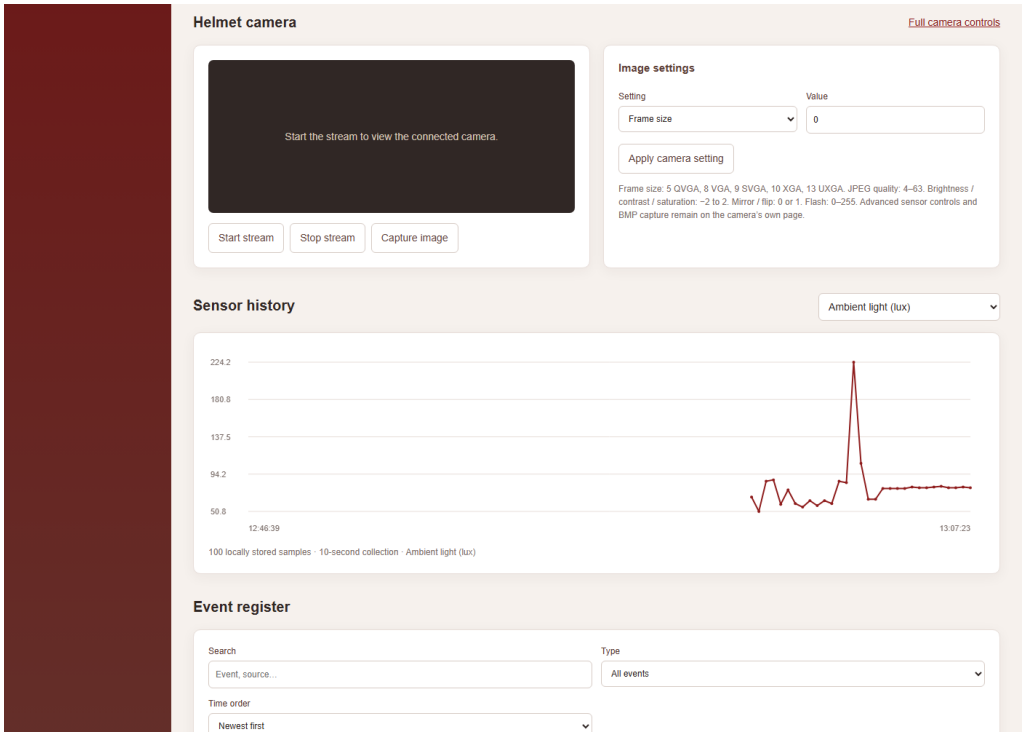


Figure 4.10: Camera panel and the stored light history of the same session.

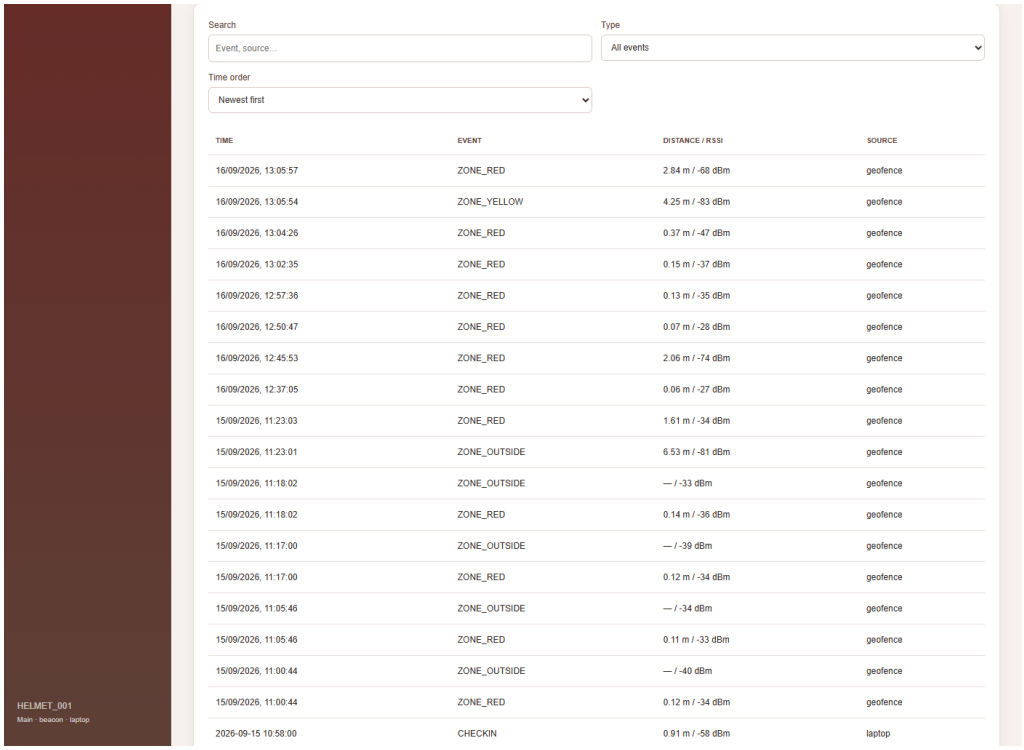


Figure 4.11: Event register and the connection settings page of the host application.

4.6 Cost Analysis

Cost was one of the fixed constraints in Section 3.2 and it is worth reporting properly, because a safety device that cannot be afforded in quantity is being designed for a paper and not for a site. Table 4.3 lists the parts against the prices published by Cytron, a Malaysian supplier, as they stood on 16 September 2026. Three lines carry no listing at that supplier, hence the MQ-7, the BH1750 and the helmet shell keep an indicative figure and the table says so on the row itself. None of these is a quotation, and a purchase at volume will move the total downward.

The total is RM 246.60 for the complete set with all sensors, which is the point of the exercise. A commercial proximity warning installation of the kind reported by Teizer et al. (2010) required dedicated hardware at a scale a small contractor would not fund, when the parts here are the sort a technical college keeps in a drawer. Cost is where this class of project has genuinely changed.

Three observations follow from the table. The first is that the three controllers together account for a smaller share of the total than the sensors do, hence adding a second helmet to the deployment costs less than the first helmet did, since the receiver and the computer are already paid for. The marginal cost per additional worker is roughly the helmet set alone. The second is that the force sensor alone costs more than both gas elements together, when the two MQ elements are the parts with the shortest useful life and the ones that will be replaced first, which is discussed in Section 4.7. The third is that the computer is not counted in the table at all, because the dashboard is designed to run on a machine the site already owns.

There are costs the table does not carry and they should not be hidden. Assembly time is real and a hand wired prototype takes several hours to build and to debug. A printed circuit board would remove most of that at a setup cost that only makes sense beyond a handful of units. Calibration gas, an instrumented head form and a reference lux meter are needed for the verification in Section 4.5 and none of them is owned by the project, which is itself a cost that has simply not been paid yet. Lastly the cloud tier is free at this volume and would not stay free at a site scale, hence

a deployment of any size should assume the local database is the record and the cloud is convenience.

Figure 4.12 splits the same bill of materials by subsystem using the prices listed in Table 4.3. Sensors account for the largest share, which is the opposite of what a reader might expect from a three controller design. The consequence is that a second helmet costs close to the first, when the receiver and the host are already paid for and only the helmet set repeats.

Table 4.3: Bill of Materials at Cytron Listed Prices, 16 September 2026

Item	Quantity	Unit price (MYR)	Listing	Subsystem
NodeMCU ESP32 board	2	8.50	Cytron, NodeMCU ESP32 Only	Main and beacon
ESP32 camera module	1	58.00	Cytron, ESP32 Camera with USB	Camera
MQ-7 carbon monoxide sensor module	1	14.00	Not listed; indicative, discontinued at Cytron	Gas
MQ-135 air quality sensor module	1	8.40	Cytron, MQ135 module	Gas
MPU6050 six-axis inertial module	1	10.50	Cytron, GY-521	Impact
BH1750 ambient light module	1	8.00	Not listed; indicative, not carried by Cytron	Illumination
DHT22 temperature and humidity module	1	15.50	Cytron, DHT22 breakout	Heat index
FSR402 force sensitive resistor	1	55.00	Cytron, 0.5 inch sensor	Presence
LEDs, buzzer, resistors and wiring	1 set	13.20	Cytron, LED kit, buzzer, jumpers	Outputs
7.4 V lithium polymer pack	1	25.00	Cytron, 7.4V 900mAh	Power
Safety helmet shell	1	22.00	Not listed; indicative, not carried by Cytron	Mechanical
Total for one complete set, both controller boards counted		246.60		Excludes the host computer

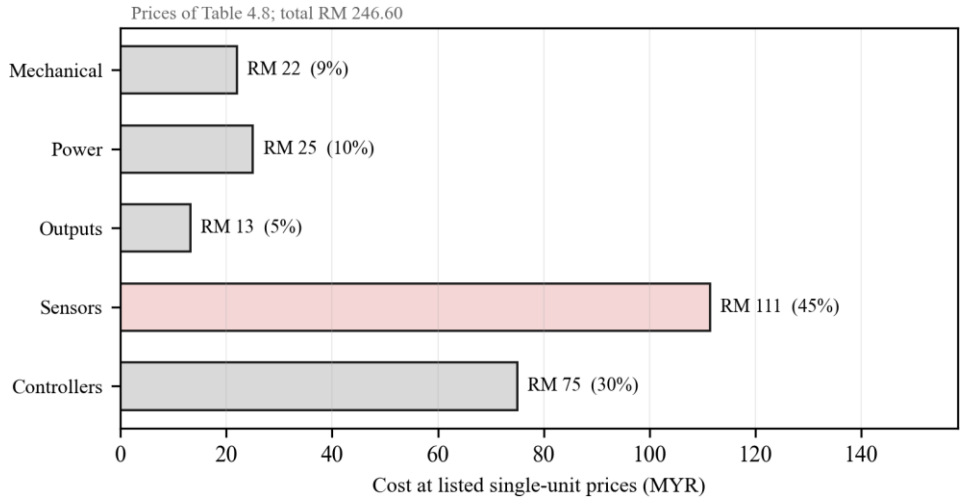


Figure 4.12: Cost by subsystem, from the prices listed in Table 4.3.

4.7 Sustainability Analysis

Sustainability for a device of this kind is decided less by its power draw than by how often it has to be thrown away, and the two MQ gas elements are the weakest part on that measure. A heated tin dioxide sensor consumes power continuously because its element must stay hot, and it drifts over its service life to the point where compensation becomes a research problem in itself (Jiang et al., 2021). A drifted element that can no longer be trusted is replaced, hence the consumable in this design is not the battery but the gas sensor.

Several choices in the build do work in the right direction. The record is held locally in SQLite and the cloud branch is secondary, which means the system continues to function during an outage and does not require a permanently powered connection to remain useful. Processing is being done at the edge, since every threshold decision, the warning arbitration and the zone decision are computed on the controllers themselves and only summaries travel to the network. A design that streamed raw samples to a server for the same decisions would move the same information at far greater cost in radio time and in energy.

Against that, several choices are clearly wasteful and the report will name them. The resistor divider draws current continuously across the pack whenever the battery is connected, as described in Section 3.18. The telemetry cycle uploads on a fixed ten second timer whether or not any value has changed, when an upload triggered by a meaningful change would carry the same information for a fraction of the traffic. The camera runs a continuous stream at a fixed frame size rather than capturing on an event.

The end of life picture is the one an engineering report usually skips. Three ESP32 boards, a camera module, six sensors and a lithium pack are electronic waste at the end of the prototype, and the lithium pack in particular must not go into general refuse. Modules mounted on headers can be recovered and reused, which is one practical argument for headers over soldered joints on a student build. The helmet shell itself is unmodified and remains a helmet.

There is a sustainability argument on the other side of the ledger that should not be overstated. A device that reduces the number of injuries avoids the material and human cost of those injuries, which dwarfs the footprint of the electronics. That argument is sound in principle. It cannot be claimed here, because no field trial has been run and no injury reduction has been measured.

4.8 Alignment with the Sustainable Development Goals

The project connects to three of the United Nations Sustainable Development Goals (United Nations, 2015), and a fourth through the discussion in Section 4.7. The connection is being described as an alignment of intent and not as a measured contribution, since a prototype that has not been deployed cannot evidence an outcome against any indicator.

Goal 3, good health and well being, is the closest fit. Target 3.9 concerns the reduction of deaths and illnesses from hazardous chemicals and from air pollution and contamination, and the helmet's gas channels and heat index warning address exactly

that class of exposure at the point where the worker breathes. Goal 8 sits alongside it through target 8.8 on protecting labour rights and promoting safe working environments for all workers, which the attendance and boundary functions serve in a narrower way, since a record of who was inside a zone and for how long is the raw material of any safe working investigation.

Goal 9 covers industry, innovation and infrastructure. The relevant part is target 9.5, on enhancing scientific research and upgrading technological capabilities, and the contribution here is modest and honest, being an integration built from commodity parts and documented well enough that another student can rebuild it. Which is the level a final year project should claim.

Goal 10, reduced inequalities, is the one most often asserted carelessly in reports of this kind, hence the care taken here. The argument is about cost. Occupational monitoring of this sort exists commercially and is bought by large employers, when the small contractors who employ a large share of construction labour in the region do not buy it. A build costing a few hundred ringgit changes who can afford to monitor, and that is a distributional claim rather than a technical one.

Goal 12 on responsible consumption and production is engaged mainly through the weaknesses set out in Section 4.7. The consumable gas elements, the continuously drawing divider and the fixed interval uploads are the places where the design consumes more than it needs to. Naming them is the contribution to Goal 12 at this stage; fixing them is future work and is listed in Section 5.2.

A last point on all of these. Alignment is not achievement. None of the goals above is being advanced by a prototype sitting on a bench, and the honest statement is that the design is pointed in these directions and would need a deployment and a measurement programme before any claim could be made. Table 4.4 sets each goal beside the part of the system that touches it, and beside what would have to be measured before the alignment could be called a result.

Table 4.4: Alignment with the United Nations Sustainable Development Goals

Goal	Target engaged	Feature of this work	Status
3 Good health and well-being	3.9	Gas channels and heat index warning at head height	Aligned, not measured
8 Decent work and economic growth	8.8	Boundary and attendance record for a safe working investigation	Aligned, not measured
9 Industry, innovation and infrastructure	9.5	Reproducible integration built from commodity parts	Partly demonstrated
10 Reduced inequalities	10.2	Low build cost places monitoring within reach of small contractors	Argued from cost only
12 Responsible consumption and production	12.5	Consumable gas elements and continuous drains identified	Identified, not fixed

4.9 Recorded Dashboard Behaviour

This section records what the host application shows while the assembled helmet runs. The readings are live values from the acquisition chain. They are not calibrated measurements, so they show that each channel produces a value and that the dashboard displays it, and nothing about accuracy. The calibration that would turn these readings into measurements is listed in Section 4.5.

The sensor history page stores about one hundred recent samples for each channel on a fixed ten second collection interval. Figure 4.13 shows the overall gas history for one session. The trace rises when a source is brought near the gas sensors, holds while the source is present, and falls back towards the baseline when it is removed. The shape follows the input as expected. The vertical axis is the relative gas index from Section 3.6 and not a concentration, so the height of the trace cannot be read in parts per million until the calibration in Section 4.5 is done.

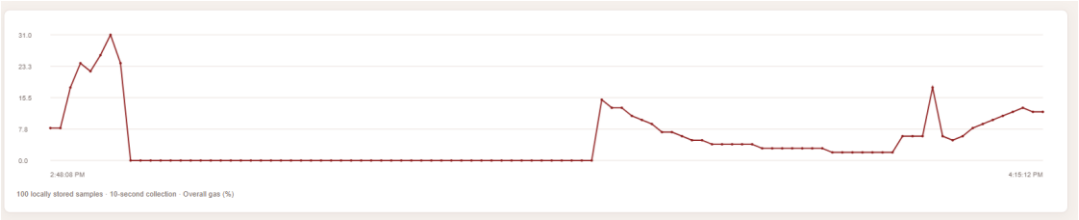


Figure 4.13: Overall gas index history for one recorded session.

The geofencing panel resolves the filtered signal strength into a zone. Figure 4.14 shows the three states captured while a person carried the helmet towards and away from the beacon. In the red state the estimated distance is about 1.8 m and the panel reports RED, VERY NEAR. In the yellow state the estimated distance is about 4.6 m and the panel reports YELLOW, NEAR. In the outside state the estimated distance is about 7.1 m and the panel reports OUTSIDE. The zone changes in the expected order as the distance increases. The distance itself depends on the measured power and the path loss exponent, and both are still default values, so the boundary distances are indicative until the radio is calibrated in Section 4.5.



Figure 4.14: Geofence zones at increasing distance. Left: red, about 1.8 m. Centre: yellow, about 4.6 m. Right: outside, about 7.1 m.

The host also raises a full screen warning when the helmet reports a danger state. Figure 4.15 shows this overlay, which the operator must acknowledge before the page returns to normal. The overlay is driven by the same priority controller that drives the helmet outputs, so what the operator sees on the host matches the state the helmet acted on.

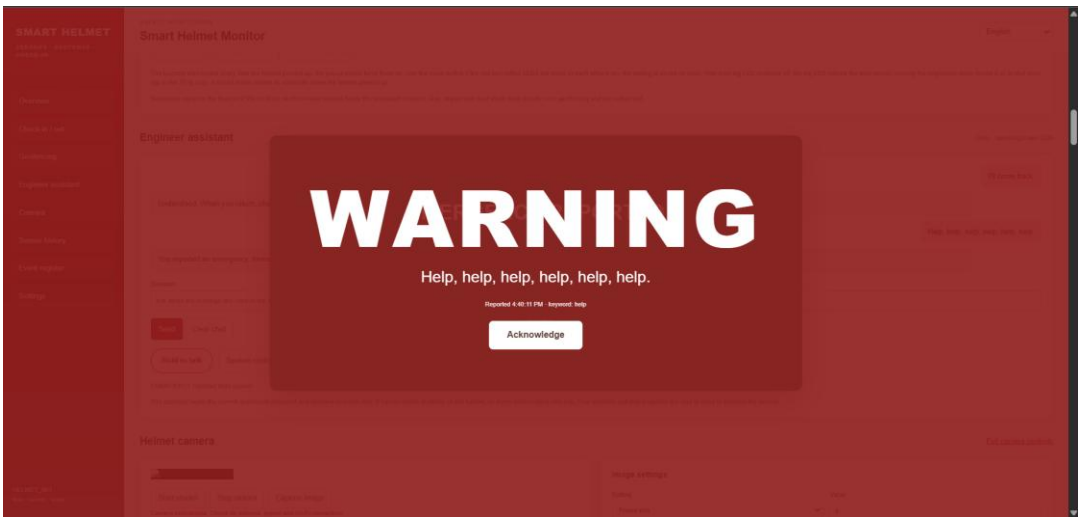


Figure 4.15: Full screen warning overlay on the host application.

The priority controller was checked against every combination of the four warning sources. Gas is considered first, then impact, then a valid high heat index, and a geofence indication reaches the shared outputs only when the other three are inactive. The overview page names the active source in its alert card, so the selected source can be read directly for each combination. Figure 4.16 shows one case where gas, impact, temperature and geofence are all active at once. The gas channels read 28 percent and 43 percent, the impact card shows an alert, and the geofence card shows the near zone, yet the alert card reports GAS. This is the highest priority source, which is the designed behaviour. Recording the full set of combinations checks that the order holds and that a lower priority source never overrides a higher one on the shared LEDs and buzzer.

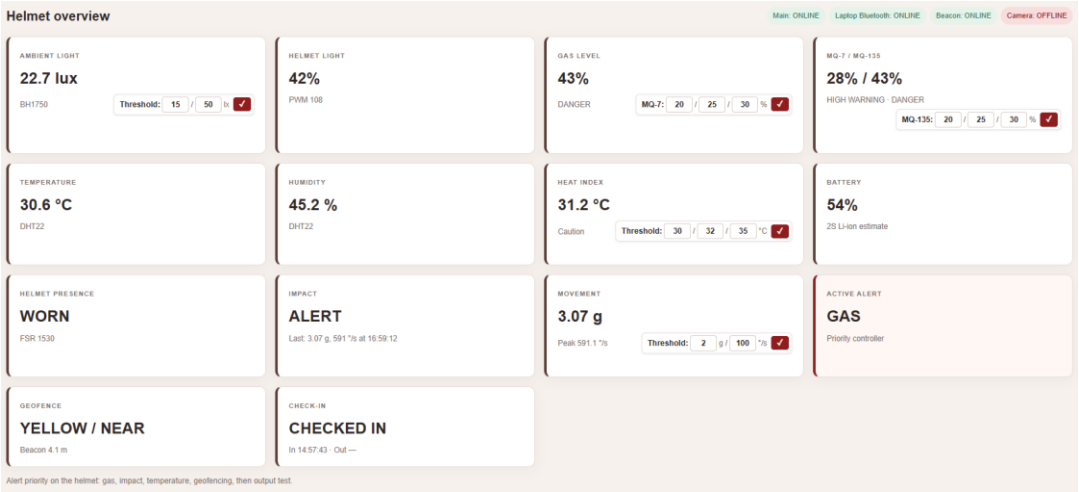


Figure 4.16: Overview page with gas, impact, temperature and geofence active together. The alert card reports GAS as the highest priority source.

One point follows from this test that matters for a safety device. The controller resolves a fixed order and it does not merge warnings. A worker who is inside a restricted zone and is also exposed to gas will see the gas warning and not the zone warning, because gas has the higher priority. This keeps the most dangerous condition visible on a single buzzer, which is the intended trade off, but it also means the lower priority condition is not shown at the same time. Whether that is acceptable for a given site is a decision for the site and not for the firmware, as noted in the problem statement.

4.10 Threats to Validity

Any claim in this report should be read against the class of evidence that supports it, and this section sets out the threats to validity in the order that they matter.

The largest is straightforward. No physical measurement exists. Every behavioural claim rests either on reading the source or on the Python suite, and the Python suite exercises the host against substituted devices, which means it proves that the host handles a response correctly and proves nothing about whether a real sensor produces that response. A reader who takes Section 4.4 as evidence that the helmet

works on a head would be reading it wrongly. What Section 4.4 evidences is that the coordination logic behaves as designed when the inputs behave as specified.

The second threat is that most constants are assumed. The gas baseline and raw maximum, the presence thresholds at 1100 and 700 counts, the measured power at -59 dBm and the exponent at 2.7 are all compile time or default values, and not one of them was fitted to the assembled device. Impact is the exception and is stated separately. For impact detection, the 2.5 g acceleration threshold is supported by Raghunath and Ghaffar (2025), while the 250 deg/s gyroscope threshold and the approximately 300 ms confirmation timing were selected through prototype motion testing. These values are engineering trigger settings for the prototype and have not yet been formally validated using an instrumented impact test setup. They are plausible starting points taken from the sensor documentation and from common practice. The system will run with them. Whether it runs correctly with them is precisely the open question in Section 4.5.

Third, the test suite is small. Twenty-eight tests covering the host is enough to catch the specific regressions they were written for and is not a coverage claim, and no automated test exists for either firmware program, since neither can be executed off the target without a simulator that the project does not have. The firmware evidence is therefore reading, and reading is weaker than execution.

Fourth, the timings reported in Section 4.3 are the durations of the test run itself. A suite that completes in under a second says something about the test harness. It says nothing about how quickly a warning reaches a wearer, and the report does not use it that way anywhere.

Lastly there is a threat from the author's own position. The design decisions, the source and the report are all the work of one person, hence a misunderstanding in the source propagates into the report without anything to catch it. The evidence ledger in Appendix B was built to reduce that risk by tying each claim to a file and a line range, so that a reader can check the claim against the source instead of against the author.

4.11 Interpretation and Limitations

All twenty-eight methods passed, but this count should not be expressed as overall system accuracy. The suite contains no measured code-coverage result and no population of physical trials. In particular, its substituted responses bypass the sensor, radio and firmware execution paths. The software evidence is strongest for host record handling and selected controls. The source inspection establishes how the other functions are programmed, including their thresholds and state transitions, rather than how accurately those functions perform under field conditions.

Several limits follow from the implementation itself. The receiver retains a bounded event ring, and Main exposes its latest impact event rather than an unlimited sequence of impacts. A sufficiently long interruption or a burst of events may therefore leave the host without every intermediate event. No loss rate has been measured. Similarly, RSSI-based distance changes with the radio environment, and the gas indices have not been calibrated against concentrations. These limits prevent a claim that the prototype has demonstrated reliable hazard detection across construction-site conditions. They also identify the measurements needed before such a claim could be assessed.

CHAPTER 5

CONCLUSIONS AND RECOMMENDATIONS

5.1 Conclusions Against Objectives

The work set out to build helmet mounted sensing, a proximity and attendance function and a host that keeps the record, as one prototype that a reader can inspect. Section 1.4 states three objectives, and the position reached against each is set out below with its evidence class attached, because an objective met in source is not an objective met on hardware.

The first objective was local monitoring. Six acquisition modules run on independent intervals with no blocking delay, the network work is being carried on a separate task, and a single arbiter owns every physical output and resolves a fixed priority. That arbiter is the strongest part of the design, since two hazards can never drive the same pin in opposite directions, which is the failure most commonly left in a build of this kind. The objective is met in source and rests on reading. What has not been shown is the timing behaviour on the assembled board under radio load, nor whether any threshold inside it is set correctly.

The second objective was proximity and attendance. The receiver recognises the configured tag, derives a zone from a filtered distance, applies the selected operating mode and retains its time settings across a restart, when the host will push a clock of its own if the receiver reports itself unsynchronised. Duplicate suppression and the manual clock override are exercised directly by the Python suite, hence this

objective carries execution evidence and not only reading. The distance model underneath is not verified and its constants are defaults. Which limits what an attendance row means physically, even though the record keeping around it behaves correctly.

The third objective was the operator interface and the record. The dashboard shows device connectivity beside the retained measurements, exposes the permitted receiver settings and the camera controls, and the local database is authoritative with retries and duplicate suppression underneath it. All of that is tested. The qualification is security, since certificate verification is disabled in the firmware and the attendance identity rests on an advertised name that can be imitated, hence the record is reliable against loss and is not reliable against a determined attacker.

Physical validation is not one of the three objectives and formal controlled validation has not yet been completed for the assembled helmet. Preliminary prototype motion testing was used during impact-subsystem development to select the 250 deg/s gyroscope threshold and the approximately 300 ms confirmation timing. These preliminary tests support parameter selection only and should not be interpreted as evidence of injury-detection accuracy, sensitivity, specificity or field reliability. Section 4.5 holds the prepared tables and Appendix E holds the procedures, and until they are filled the project is an integration whose logic has been checked and whose physics has not.

Overall, the system-integration logic was implemented, while formal physical validation remains incomplete. Preliminary impact motion testing was used for threshold and confirmation-window tuning, but broader controlled measurement and field evaluation are still required.

5.2 Recommendations

The recommendations are ordered by what blocks the most, and the first four are prerequisites for any claim about performance rather than improvements to it.

Calibrate before anything else. The gas baseline and raw maximum must be taken from the actual elements against a known source, the measured power and path loss exponent must be fitted from readings at known separations in the room where the system will run, and the presence thresholds must be set from the assembled harness with a wearer in it. Until this is done every threshold in the system is a guess that happens to be written in a constant, hence no result obtained before calibration is worth recording.

Second, secure the transport. Certificate verification is disabled in the Main firmware and the fix is small, being either a pinned root certificate compiled into the image or a bundled certificate store. This is a few lines of work and it removes a defect that an examiner will find immediately.

Third, bind the tag identity. The receiver accepts any device advertising the name TAG001 with the expected service, which is the exposure demonstrated by Kim, Lee and Paek (2018). Moving to BLE bonding with a resolvable private address would make the receiver verify a key instead of a string, and the attendance record would then mean something it currently does not.

Fourth, run the measurements in Section 4.5 and fill the tables. The procedures are already fixed, so the work is bench time and not design.

Beyond those, several improvements are worth the effort. The impact slot should become a small queue so that two events inside one polling interval both reach the host with their magnitudes intact. The battery divider should be switched or raised in value, which removes the continuous drain described in Section 3.18. The camera should capture a still when an alert is raised and store it beside the event row, since an event with no image is harder to review afterwards. Telemetry should upload on a

meaningful change instead of a fixed ten second timer, which reduces traffic and energy together.

Two larger directions are worth naming and neither belongs in this project. A second receiver would turn a range into an intersection and would let the system report a position instead of a zone. A dashboard with real authentication would let the host leave the assumption of a single trusted machine on a private network, which is the assumption everything in Section 3.16 currently rests on.

REFERENCES

Abdullah, ZO, Celik, T & Celik, T 2025, ‘Proposing an affordable real-time camera-based safety and quality management framework for construction industries in developing countries’, Buildings, vol. 15, no. 2, article 156.

Campero-Jurado, I, Marquez-Sanchez, S, Quintanar-Gomez, J, Rodriguez, S & Corchado, JM 2020, ‘Smart Helmet 5.0 for Industrial Internet of Things using artificial intelligence’, Sensors, vol. 20, no. 21, article 6241.

Chen, Y.-C., Yu, T.-C. and Li, C.-Y. (2025) 'An Automated Detection Platform Against TLS Vulnerabilities for Wi-Fi IoT Devices', 2025 IEEE Conference on Dependable and Secure Computing (DSC), pp. 1-2. doi:10.1109/DSC65356.2025.11260871.

Choi, B., Hwang, S. and Lee, S. (2017) 'What drives construction workers' acceptance of wearable technologies in the workplace?: Indoor localization and wearable health devices for occupational safety and health', Automation in Construction, 84, pp. 31-41. doi:10.1016/j.autcon.2017.08.005.

Dubai Municipality 2019, Technical guidelines for management of heat stress at work, DM-HSD-GU38-MHSW2, Dubai Municipality, Dubai.

Espressif Systems (n.d.a) ESP32 Series Datasheet. Version 5.3. Available at: https://documentation.espressif.com/esp32_datasheet_en.html (Accessed: 6 September 2026).

Espressif Systems (n.d.b) RF Coexistence. ESP-IDF Programming Guide, v5.0. Available at: https://docs.espressif.com/projects/esp-idf/en/v5.0/esp32/api-guides/coexist.html (Accessed: 6 September 2026).

Fernandez-Garcia, R. and Gil, I. (2017) 'An Alternative Wearable Tracking System Based on a Low-Power Wide-Area Network', Sensors, 17(3), 592. doi:10.3390/s17030592.

Filus, K., Nowak, S., Domanska, J. and Duda, J. (2022) 'Cost-effective filtering of unreliable proximity detection results based on BLE RSSI and IMU readings using smartphones', Scientific Reports, 12(1), 2440. doi:10.1038/s41598-022-06201-y.

Google (2026) Saving Data. Firebase Realtime Database REST documentation. Available at: https://firebase.google.com/docs/database/rest/save-data (Accessed: 6 September 2026).

Han, K. and Zeng, X. (2022) 'Deep Learning-Based Workers Safety Helmet Wearing Detection on Construction Sites Using Multi-Scale Features', IEEE Access, 10, pp. 718-729. doi:10.1109/ACCESS.2021.3138407.

Jiang, Z., Xu, P., Du, Y., Yuan, F. and Song, K. (2021) 'Balanced Distribution Adaptation for Metal Oxide Semiconductor Gas Sensor Array Drift Compensation', Sensors, 21(10), 3403. doi:10.3390/s21103403.

Kim, M., Lee, J. and Paek, J. (2018) 'Neutralizing BLE Beacon-Based Electronic Attendance System Using Signal Imitation Attack', IEEE Access, 6, pp. 77921-77930. doi:10.1109/ACCESS.2018.2884488.

Kim, Y. and Choi, Y. (2022) ‘Smart Helmet-Based Proximity Warning System to Improve Occupational Safety on the Road Using Image Sensor and Artificial Intelligence’, International Journal of Environmental Research and Public Health, 19(23), 16312. doi: 10.3390/ijerph192316312.

Kim, Y., Baek, J. and Choi, Y. (2021) ‘Smart Helmet-Based Personnel Proximity Warning System for Improving Underground Mine Safety’, Applied Sciences, 11(10), 4342. doi: 10.3390/app11104342.

Lee, P, Kim, H, Zitouni, MS, Khandoker, A, Jelinek, HF, Hadjileontiadis, L, Lee, U & Jeong, Y 2022, ‘Trends in smart helmets with multimodal sensing for health and safety: Scoping review’, JMIR mHealth and uHealth, vol. 10, no. 11, article e40797.

Main.zip (n.d.) Smart Helmet implementation archive [computer program]. Supplied project files; inspected 16 September 2026. Source identity and file mappings are retained in the supporting evidence ledger.

Maratea, A., Salvi, G. and Gaglione, S. (2019) 'Bagging to Improve the Calibration of RSSI Signals in Bluetooth Low Energy (BLE) Indoor Distance Estimation', 2019 15th International Conference on Signal-Image Technology and Internet-Based Systems (SITIS). doi:10.1109/SITIS.2019.00107.

Marquez-Sanchez, S, Campero-Jurado, I, Herrera-Santos, J, Rodriguez, S & Corchado, JM 2021, 'Intelligent platform based on smart PPE for safety in workplaces', Sensors, vol. 21, no. 14, article 4652.

Marsh, S.M. and Jackson, L.L. (2013) 'A comparison of fatal occupational injury event characteristics from the Census of Fatal Occupational Injuries and the National Vital Statistics System', Journal of Safety Research, 46, pp. 119-125. doi:10.1016/j.jsr.2013.05.004.

Merchan-Cruz, EA, Moveh, S, Pasha, O, Tocelovskis, R, Grakovski, A, Krainyukov, A, Ostrovenecs, N, Gercevs, I & Petrovs, V 2025, 'Smart safety helmets with integrated vision systems for industrial infrastructure inspection: A comprehensive review of VSLAM-enabled technologies', Sensors, vol. 25, no. 15, article 4834.

Michaeli, L., Michalko, P. and Saliga, J. (2008) 'Unified ADC nonlinearity error model for SAR ADC', Measurement, 41(2), pp. 198-204. doi:10.1016/j.measurement.2006.10.004.

Minhas, MI et al. 2026, 'A low-cost smart helmet with accident detection and emergency response for bike riders', Journal of Sensor and Actuator Networks, vol. 15, no. 1, article 20.

Raghunath, S. and Ghaffar, S.H. (2025) 'Developing an IoT-Enabled Smart Helmet for Worker Safety: Technical Feasibility and Business Model', Safety, 11(3), 89. doi:10.3390/safety11030089.

Ramanathan, N.L. and Belding, H.S. (1973) 'Physiological Evaluation of the WBGT Index for Occupational Heat Stress', American Industrial Hygiene Association Journal, 34(9), pp. 375-383. doi:10.1080/0002889738506866.

Ramirez, R., Huang, C.-Y., Liao, C.-A., Lin, P.-T., Lin, H.-W. and Liang, S.-H. (2021) 'A Practice of BLE RSSI Measurement for Indoor Positioning', Sensors, 21(15), 5181. doi: 10.3390/s21155181.

Saikia, M.J. and Alkhader, A.S. (2024) 'Smart Textile Impact Sensor for e-Helmet to Measure Head Injury', Sensors, 24(9), 2919. doi:10.3390/s24092919.

Sharma, M, Suri, NM & Kant, S 2022, 'Analyzing occupational heat stress using sensor-based monitoring: A wearable approach with environmental ergonomics perspective', International Journal of Environmental Science and Technology, vol. 19, no. 11, pp. 11421-11434.

Shema, M. (2010) 'Cross-Site Request Forgery', in Seven Deadliest Web Application Attacks. Burlington, MA: Syngress, pp. 27-45. doi:10.1016/B978-1-59749-543-1.00002-5.

Subedi, S. and Pyun, J.-Y. (2020) 'A Survey of Smartphone-Based Indoor Positioning System Using RF-Based Wireless Technologies', Sensors, 20(24), 7230. doi:10.3390/s20247230.

Sun, D. and Tian, Y. (2022) 'Address Privacy of Bluetooth Low Energy', Mathematics, 10(22), 4346. doi:10.3390/math10224346.

Swetha, A & Devigasri, V 2025, 'Smart helmet for industrial workers safety', International Research Journal of Modernization in Engineering Technology and Science, vol. 7, no. 5.

Teizer, J., Allread, B.S., Fullerton, C.E. and Hinze, J. (2010) 'Autonomous pro-active real-time construction worker and equipment operator proximity safety alert system', Automation in Construction, 19(5), pp. 630-640. doi:10.1016/j.autcon.2010.02.009.

U.S. National Weather Service 2022, The heat index equation, Weather Prediction Center, viewed 2 May 2026.

United Nations (2015) Transforming our world: the 2030 Agenda for Sustainable Development. Resolution A/RES/70/1. New York: United Nations General Assembly. Available at: https://sdgs.un.org/2030agenda (Accessed: 9 September 2026).

Wu, X, Li, Y, Long, J, Zhang, S, Wan, S & Mei, S 2023, ‘A remote-vision-based safety helmet and harness monitoring system based on attribute knowledge modeling’, Remote Sensing, vol. 15, no. 2, article 347.

Zhao, X., McDermott, B.P. and Vanos, J.K. (2025) 'Heat index and humidex, but not other measures, identify heat stress compensability during moderate work', Temperature, 12(4), pp. 337-350. doi:10.1080/23328940.2025.2572271.

Zhengzhou Winsen Electronics Technology Co., Ltd. (2021) MQ135 Air Quality Gas Sensor. Product manual, version 1.6. Available at: https://www.winsen-sensor.com/manual/mq135.html (Accessed: 6 September 2026).

APPENDIX A: INSTALLATION AND OPERATING MANUAL

Installation and Configuration

The deployment procedure starts from the Main directory extracted from Main.zip. Keep the firmware subfolders and the Python application together so that their supplied configuration and dependency files remain available. Main and Receiver require compatible ESP32 targets, while Cam uses the AI Thinker ESP32-CAM pin definition. Install the ESP32 board package and the libraries required by Main: BH1750, MPU6050_tockn, DHT sensor library, Adafruit Unified Sensor and ArduinoJson. Record the versions selected for a build. The report does not contain a verified final build log for all three sketches.

Before connecting power, compare the assembly with Table 3.3. Main drives a single buzzer on GPIO 33, active high, and every warning branch reaches that one output. GPIO 26 is left unused in the final assembly, hence warning priority is resolved in software on one shared output. The battery ADC requires the 100 kOhm and 47 kOhm divider; the battery must not be connected directly to the ADC input. The camera uses its own pin map. Wiring and supply checks should be completed before the proposed output tests are attempted.

Enter the same network name and password in each firmware folder's NetworkConfig.h. Compile and upload Main, Receiver and Cam to their corresponding boards, then record the addresses printed through their serial interfaces. Use those addresses if the .local names do not resolve on the computer. The computer and devices must be reachable on the shared network. A USB connection can supply receiver power and serial access while Wi-Fi carries the application controls.

For the camera module, compile and upload the latest AI Thinker ESP32-CAM sketch containing the configured Wi-Fi credentials and static local network settings. The current prototype camera uses static IP 10.249.219.150, gateway 10.249.219.22 and subnet mask 255.255.255.0. After the ESP32-CAM is powered and connected to the same configured Wi-Fi network, the camera interface can be

accessed directly through the fixed local IP address. Serial Monitor may still be used for diagnostics, but the camera does not depend on receiving a different DHCP address after each restart.

In the extracted python folder, run setup.ps1 from PowerShell. The script prepares the virtual environment, installs requirements.txt and creates config.json only if it does not already exist. Inspect that configuration before starting live operation. Set the Main, Receiver and camera addresses, including the separate camera stream URL. The supplied stream URL uses port 81. Configure Firebase only for an account and database intended for this installation. Keep credentials out of screenshots, report extracts and exported examples.

Run start.cmd to launch the application, then open the configured local address. The example configuration uses http://127.0.0.1:5000. Use preview.cmd for the synthetic demonstration view. Demonstration values must remain distinguishable from device measurements. The command sequence and configuration fields are collected in Table A.1. If setup fails, retain the terminal error and check the selected interpreter and dependency installation before changing the application source. A successful page load alone does not verify communication with each device; confirm the separate connectivity indications and current telemetry.

Table A.1: Installation Commands and Configuration

Step / field	Value or action
Working directory	The python subfolder extracted from Main.zip
Environment setup	.\setup.ps1
Edit settings	Open config.json after setup
Live launch	.\start.cmd
Synthetic preview	.\preview.cmd
main_url	http://helmet-main.local or the recorded IP
receiver_url	http://helmet-receiver.local or the recorded IP
camera_url	http://10.249.219.150
camera_stream_url	http://10.249.219.150:81/stream
host / port	Example: 127.0.0.1 / 5000
firebase_url / firebase_token	Installation-specific database and credential

Step / field	Value or action
NetworkConfig.h	NETWORK_SSID and NETWORK_PASSWORD on all three boards

Operation and Troubleshooting

During live operation, inspect the connectivity state before interpreting a sensor card. An offline device can leave a retained value on the dashboard. Check Main and Receiver separately, then compare the receiver decision with the BLE state reported by Main. This comparison helps identify whether the host has received a decision that has not yet reached the helmet. The camera panel provides image access and supported controls; it does not perform object recognition.

Select GEOFENCE for the proximity indication or TAGGING for attendance operation. The receiver's TEST mode provides output commands, but higher-priority Main conditions can replace the requested indication. Record the selected mode when checking an output. Set the worker name, identifier, role and station for TAG001 before collecting a visit. This roster is descriptive information for one configured tag, not independent proof of the wearer's identity.

For attendance, set the expected check-in and checkout hours, grace period, entry dwell and visit cooldown. Confirm receiver time before beginning a session. The default checkout delay is five continuous seconds outside the proximity zones, whereas the configured grace is applied to the overdue deadline. These delays serve different purposes. To demonstrate a later time, use the manual clock control and observe the displayed receiver time. Return to computer time when the demonstration ends. After a receiver restart, check time synchronisation again even if the saved schedule still appears correct.

Calibrate the distance model in the intended room using known separation distances and recorded RSSI observations. The supplied boundary values are starting settings, not measured guarantees of coverage. Keep simulation records labelled and separate from physical observations. A simulated distance can exercise the attendance

logic, but it does not demonstrate wireless reception at that distance. The output-test controls should likewise be recorded as test commands rather than as detected hazards.

Use the event register to inspect attendance and impact records, then download the CSV or Excel export. If an open workbook prevents replacement, close it and retry the export. The database remains the primary record source for the tested retry path. Retain the SQLite database with its associated application configuration and evidence log when archiving a run, taking care not to include private credentials in a shared copy. Table A.2 lists common symptoms and the checks that distinguish configuration problems from unverified hardware behaviour. When a fault remains, retain the actual error, device state and test conditions instead of describing the whole system as either working or failed.

Table A.2: Operational Troubleshooting

Symptom	Check / response
Page does not open	Read server terminal; confirm configured host and port.
Device offline	Confirm power, shared network and recorded IP; distinguish retained values from current telemetry.
Receiver changes; Main does not	Compare BLE freshness and command delivery; check higher-priority local alerts.
Attendance time unexpected	Check receiver clock, manual override, schedule and grace; resynchronise after restart.
Distance fluctuates	Retain raw conditions; check orientation, obstruction and calibration.
Camera unavailable	Check camera address and separate port 81 stream; capture and streaming are distinct paths.
Excel export cannot replace file	Close the workbook and retry; inspect SQLite record and error.
Cloud branch stale	Identify Main or host ownership; inspect configuration and request errors.

APPENDIX B: SOURCE AND VERIFICATION RECORDS

Appendix A provides the operating manual. Appendix B records the source and verification artefacts needed to reproduce this report's evidence trail. Table B.1 identifies their roles, including the implementation archive, test log, checked prose and document-format audit. Those artefacts allow a later reviewer to locate the basis of a statement and distinguish source-derived behaviour from executed software results.

Table B.1: Source and Verification Artefacts

Artefact	Role
Main.zip	Authoritative supplied implementation; archive hash in 03_EVIDENCE_LEDGER.csv and SOURCE_PROVENANCE.json
FYP_Report_V11.docx	User-selected formatting authority; audited SHA-256
test_execution.txt / results_notes.md	Executed test log and bounded assertion interpretation
03_EVIDENCE_LEDGER.csv / 02_CODEBASE_MAP.md	Traceability from report functions to source
references.csv	Verified primary research and official documentation
ZEROGPT_RESULTS.csv / CHUNK_FILE_HASHES.json	Observed detector results and accepted-text hashes
report_content.json / assemble_content.py / build_report.py	Reproducible content assembly
Smart_Helmet_Report.pdf / FINAL_QA.md	Rendered report and final review record

Evidence Traceability

A claim is only worth as much as the place a reader can go and check it. Table B.2 ties each technical statement in the report back to the file and the line range it came from, hence a disagreement about what the system does will be settled against the implementation and not against the author’s memory of it.

Twenty seven entries are recorded. The status column is doing the real work in that table, because reading a function and running it are not the same evidence, and the report leans on both without pretending they are equal. Where the status says implemented, the behaviour was read in the source, and twenty one of the entries sit there. Where it says exercised by test, the Python suite ran it and the outcome was observed, which is the stronger position and covers a single entry.

One entry says something else entirely. Absent by observation means the feature was looked for and was not there, hence the gap is being written down as a finding rather than quietly dropped, which is what usually happens to a function that was planned and never built.

Paths are relative to the Main directory inside the archive. Where a claim rests on more than one file the first is given here and the remainder stay in the ledger kept with the build sources.

Table B.2: Evidence Traceability from Report Claim to Source Location

Ref	Subject	Source location	Evidence status
E001	architecture	firmware/Main/Main.ino : Main.ino:18-31	Implemented
E002	ambient light	firmware/Main/Main.ino : 65-108	Implemented
E003	gas	firmware/Main/Main.ino : 110-174	Implemented
E004	gas alert	firmware/Main/Main.ino : 138-174	Configured
E005	impact	firmware/Main/Main.ino : 186-200	Implemented
E006	temperature	firmware/Main/Main.ino : 239-270	Implemented
E007	battery	firmware/Main/Main.ino : 272-297	Implemented
E008	presence	firmware/Main/Main.ino : 299-327	Implemented
E009	alert priority	firmware/Main/Main.ino : 2303-2337	Implemented
E010	firebase	firmware/Main/Main.ino : 44-63	Implemented

Ref	Subject	Source location	Evidence status
E011	tls	firmware/Main/Main.ino : 1361-1367	Implemented, limited
E012	main http	firmware/Main/Main.ino : 2339-2375	Implemented
E013	ble tag	firmware/Main/Main.ino : 2266-2301	Implemented
E014	camera	firmware/Cam/board_config.h : board_config.h:23	Implemented
E015	camera	firmware/Cam/Cam.ino : 45-64	Configured
E016	receiver scan	firmware/receiver/receiver.ino : 14-15	Implemented
E017	geofence	firmware/receiver/receiver.ino : 25	Implemented
E018	attendance	firmware/receiver/receiver.ino : 85-106	Implemented
E019	schedule	firmware/receiver/receiver.ino : 57-63	Implemented
E020	receiver api	firmware/receiver/receiver.ino : 171-195	Implemented
E021	simulation	firmware/receiver/receiver.ino : 129-148	Implemented
E022	dashboard polling	python/app.py : 64-115	Implemented
E023	storage	python/storage.py : 16-19	Implemented
E024	demo	python/app.py : 190-208	Implemented
E025	software tests	python/tests/test_system.py : 23-112	Exercised by test
E026	software tests	python/tests/test_system.py : 23-112	Verified in audit
E027	results boundary	python/demo-data/helmet.sqlite3 : file inventory	Absent by observation

APPENDIX C: FIRMWARE INTERFACES AND CONFIGURATION

The detail in this appendix would interrupt Chapter 3 if it sat there, hence it is collected here instead. Every entry comes from the source in Main.zip and can be checked against the file and line ranges in Appendix B, and where a datasheet and the source disagree the source is what has been written down.

Pin Assignments

Pin choice was not free. Every analogue input sits on ADC1 because the second converter is handed to the Wi-Fi driver while the radio is up, so a reading taken on ADC2 would fail the moment the helmet connected. That single constraint fixed the gas, battery and presence pins. GPIO 34 to 39 are input only and carry no internal pull resistors either, which is why each of them arrives through its own external divider. Table C.1 gives the complete pin map, with the electrical note that applies to each line.

Table C.1: Complete Pin Map of the Main Controller

Signal	GPIO	Direction	Electrical note
I2C data (BH1750, MPU6050)	21	Bidirectional	Bus clocked at 100 kHz
I2C clock (BH1750, MPU6050)	22	Output	Shared by both devices
Helmet lamp	25	Output	PWM 5 kHz, 8-bit, command capped at 178
MQ-7 analogue	34	Input	ADC1, input only pin
MQ-135 analogue	35	Input	ADC1, input only pin
Yellow indicator	18	Output	Driven only by the arbiter
Red indicator	19	Output	Driven only by the arbiter
Shared buzzer	33	Output	Active high; gas, impact, temperature Danger, geofence
DHT22 data	4	Bidirectional	Single wire, 2 s interval
Battery divider tap	36	Input	ADC1 VP, 100k over 47k
FSR402 tap	39	Input	ADC1 VN, 10k to ground

Receiver Control Interface

The receiver checks a parameter before it applies it, and a request that fails the check returns a 400 with a message and changes nothing at all. This matters more than it looks. A partly applied calibration would leave the distance model half old and half new, hence the receiver either takes the whole set or takes none of it. Table C.2 lists the routes, the parameters each accepts and the validation applied before anything is changed.

Table C.2: Receiver Control Interface

Route	Parameters	Validation applied
/api/status	None	Returns the full state including the last 64 events
/api/mode	value	Accepts GEOFENCE, TAGGING or TEST only
/api/settings	checkin, checkout, grace, dwell_s, cooldown_s	Times parsed as HH:MM, numeric ranges bounded, deadline recalculated
/api/time	epoch, tz	Epoch bounded to 1700000000 to 4102444800, offset to plus or minus 840
/api/calibrate	measured_power, path_loss_exponent, reference_distance_m, near_m, outer_m, hysteresis_m, auto	Each bounded per Table 3.5; near must be less than outer; auto requires a live sample
/api/roster	name, person_id, role, station	Truncated to 80, 40, 60 and 80 characters
/api/simulate	enabled, distance_m	Distance bounded 0 to 100 m; toggling clears attendance state
/api/selftest	channel	Accepts RED, YELLOW or BUZZER, and only while in TEST mode

Local Database Schema

Storage is a single SQLite file opened in write ahead logging mode. Three tables are created at startup, and the events table carries the composite deduplication key described in Section 3.16. Table C.3 gives the schema of those three tables.

Table C.3: Local Database Schema

Table	Columns	Purpose
telemetry	id, received, data	Ten second helmet snapshots stored as JSON
events	key, received, data, synced	Attendance and impact events, key is source plus boot id plus sequence
metadata	key, data	Reserved key and value store

APPENDIX D: SELECTED SOURCE LISTINGS

The listings below are reproduced verbatim from the implementation in Main.zip and carry the algorithms that the report relies on most heavily. They are included here because the project guidelines allow computer programme listings to be placed in an appendix when they would be too lengthy to sit in the text. Layout has been closed up in the Arduino listings, since the original source places one argument per line, and no token, constant or operator has been altered. The complete source remains in Main.zip, and the file and line ranges recorded in Appendix B allow every listing to be located in the original.

D.1 Ambient light to lamp command mapping (Main.ino)

The inverse quadratic response and the 178 command ceiling described in Section 3.5.

```
int calculateMainLEDPWM(float lux) {
  if (lux <= MIN_LUX) {
    return MAX_LED_PWM;
  }
  if (lux >= MAX_LUX) {
    return 0;
  }
  float normalized = (lux - MIN_LUX) / (MAX_LUX - MIN_LUX);
  float outputRatio = (1.0 - normalized) * (1.0 - normalized);
  int pwm = round(MAX_LED_PWM * outputRatio);
  return constrain(pwm,0,MAX_LED_PWM);
}
```

D.2 Gas span mapping and classification (Main.ino)

The dimensionless percentage of span and the three warning boundaries discussed in Section 3.6 and Section 2.4.

```
int rawToGasPercent(int raw,int baseline,int rawMax) {
  if (rawMax <= baseline) {
    return 0;
  }
```

```
  int pct = (int)((long)(raw - baseline) * 100 / (rawMax - baseline));
  pct = constrain(pct,0,100);
  return pct;
}
WarnState classifyGas(int percentage) {
  if (percentage >= PCT_DANGER) {
    return WARN_DANGER;
  }
  if (percentage >= PCT_BLINK) {
    return WARN_BLINK;
  }
  if (percentage >= PCT_YELLOW) {
    return WARN_YELLOW;
  }
  return WARN_SAFE;
}
```

D.3 Two-stage impact detection (Main.ino)

Acceleration candidate, gyroscope confirmation window, alert duration and cooldown, as described in Section 3.7.

```
void updateImpactDetection(unsigned long now) {
  if (now - lastImpactSample < IMPACT_SAMPLE_INTERVAL_MS) {
    return;
  }
  lastImpactSample = now;
// Keep sensor values updated even while alerting.
  mpu6050.update();
  float ax = mpu6050.getAccX();
  float ay = mpu6050.getAccY();
  float az = mpu6050.getAccZ();
  float gx = mpu6050.getGyroX();
  float gy = mpu6050.getGyroY();
  float gz = mpu6050.getGyroZ();
```

```
  float Ares = sqrt(ax * ax + ay * ay + az * az);
  float Gres = sqrt(gx * gx + gy * gy + gz * gz);
  accelerationResultantG = Ares;
  gyroResultantDPS = Gres;
// Existing alert duration.
  if (impactAlertActive) {
    if (now - impactAlertStart >= ALERT_DURATION_MS) {
      stopImpactAlert();
    }
    return;
  }
// Existing cooldown logic.
  if (now - lastImpactAlertTime < COOLDOWN_MS) {
    return;
  }
// Stage 1:
// acceleration threshold
  if (!impactCandidateActive && Ares > ACC_THRESHOLD_G) {
    impactCandidateActive = true;
    impactCandidateStart = now;
    Serial.println("[IMPACT] Acceleration threshold exceeded. Waiting for gyro
confirmation...");
  }
```

D.4 Heat index computation (Main.ino)

The Rothfusz regression applied to the DHT22 readings, described in Section 3.8.

```
float calculateHeatIndexC(float tempC,float rh) {
  float tempF = tempC * 9.0 / 5.0 + 32.0;
  float hiF = -42.379 + 2.04901523 * tempF + 10.14333127 * rh - 0.22475541 * tempF
* rh - 0.00683783 * tempF * tempF - 0.05481717 * rh * rh + 0.00122874 * tempF *
tempF * rh + 0.00085282 * tempF * rh * rh - 0.00000199 * tempF * tempF * rh * rh;
  return (hiF - 32.0) * 5.0 / 9.0;
}
```

D.5 Shared output arbitration (Main.ino)

The single arbiter that owns every physical output, and the fixed priority explained in Section 3.10.

```
void updateAlertOutputs(uint32_t now) {
 bool red=false,yellow=false,buzz=false,impact=false;
 activeAlert="NONE";
 // Any gas warning owns the indicators, including its original silent levels.
 if(currentGasWarn != WARN_SAFE) {
  activeAlert="GAS";
  if(currentGasWarn==WARN_YELLOW) yellow=true;
  else if(currentGasWarn==WARN_BLINK) red=yellow=(now/300)%2;
  else {red=true; buzz=(now/200)%2;}
 } else if(impactAlertActive) {
  activeAlert="IMPACT"; red=true; buzz=true; impact=true;
 } else if(temperatureReadingValid && heatIndexC>34.0f) {
  // Reuse the source's existing Caution / Warning / Danger boundaries.
  activeAlert="TEMPERATURE"; yellow=true;
  if(heatIndexC>39.0f) {red=(now/300)%2; buzz=(now/600)%2;}
  if(heatIndexC>45.0f) {red=true; buzz=true;}
 } else {
  xSemaphoreTake(bleLock,portMAX_DELAY);
  String cmd=receiverCommand;
  uint32_t elapsed=now-patternStarted;
  bool fresh=receiverUpdated && now-receiverUpdated<10000;
  xSemaphoreGive(bleLock);
  if(fresh) {
   if(cmd=="ZONE:RED") {activeAlert="GEOFENCE";red=true;buzz=true;}
   else if(cmd=="ZONE:YELLOW")
{activeAlert="GEOFENCE";yellow=true;buzz=(now/150)%2;}
   else if(cmd=="ALARM:ON")
{activeAlert="OVERDUE";red=yellow=buzz=(now/300)%2;}
   else if(cmd=="CHECKIN:OK" && elapsed<1200)
```

```
{activeAlert="CHECKIN";red=yellow=true;buzz=elapsed<180;}
     else if(cmd=="CHECKIN:LATE" && elapsed<1400)
{activeAlert="LATE";red=yellow=true;buzz=elapsed<180 || (elapsed>=360 &&
elapsed<540);}
     else if(cmd.startsWith("TEST:") && elapsed<2000)
{activeAlert="TEST";red=cmd=="TEST:RED";yellow=cmd=="TEST:YELLOW";bu
zz=cmd=="TEST:BUZZER";}
    }
  }
  digitalWrite(PIN_LED_RED,red); digitalWrite(PIN_LED_YELLOW,yellow);
  digitalWrite(PIN_GAS_BUZZER,buzz);
  digitalWrite(IMPACT_BUZZER_PIN,HIGH);   // unused: one buzzer on GPIO 33
}
```

D.6 Distance model and zone decision (receiver.ino)

The log distance conversion, the exponential filter and the hysteresis boundaries of Section 3.12.

```
float distanceNow(){return simulated?simDistance: reference*powf(10,(measured-
filteredRSSI)/(10*exponent));}

class ScanCallbacks:public BLEAdvertisedDeviceCallbacks{
  void onResult(BLEAdvertisedDevice d) override{
    if(!d.haveName() || d.getName()!="TAG001" || !d.haveServiceUUID()
|| !d.isAdvertisingService(BLEUUID(TAG_SERVICE)))return;
    xSemaphoreTake(stateMutex,portMAX_DELAY);

rawRSSI=d.getRSSI();filteredRSSI=lastSeen? .35f*rawRSSI+.65f*filteredRSSI:rawR
SSI;
    lastSeen=millis();address=d.getAddress().toString();
    xSemaphoreGive(stateMutex);
  }
};
```

```
void evaluateState(){
  uint32_t ms=millis();float dist=distanceNow();
  detected=simulated || (lastSeen && ms-lastSeen<2500);
  // Separate entry/exit boundaries suppress threshold chatter.
  if(!detected)zone="OUTSIDE";
  else if(zone=="RED / VERY NEAR" && dist<nearM+hysteresis)zone="RED /
VERY NEAR";
  else if(dist<nearM)zone="RED / VERY NEAR";
  else if(dist<outerM || (zone!="OUTSIDE" &&
dist<outerM+hysteresis))zone="YELLOW / NEAR";
  else zone="OUTSIDE";
  if(mode=="GEOFENCE"){command=zone=="RED / VERY
NEAR"?"ZONE:RED":zone=="YELLOW /
NEAR"?"ZONE:YELLOW":"ZONE:OUT";return;}
  if(mode=="TEST"){command=(int32_t)(testUntil-
ms)>0?testCommand:"IDLE";return;}
```

D.7 Attendance state machine (receiver.ino)

Dwell, cooldown, late classification, checkout and the overdue alarm, described in Section 3.13.

```
  command="IDLE";
  if(!timeSynced)return;
  bool inside=zone!="OUTSIDE";
  if(inside && (!checkedIn || checkedOut)){
    if(lastCheckoutMs && ms-lastCheckoutMs<cooldownMs)return;
    if(!entrySince)entrySince=ms;
    if(ms-entrySince>=dwellMs){

checkedIn=true;checkedOut=false;overdueAlarm=false;checkInAt=nowLocal();check
OutAt=0;outsideSince=0;
      late=(checkInAt%86400)/60>checkInMinute;recalculateDeadline();
```

```
addEvent(late?"CHECKIN_LATE":"CHECKIN");confirmation=late?"CHECKIN:LATE":"CHECKIN:OK";pendingConfirmationAt=ms;
    }
  }else if(!inside)entrySince=0;
  if(checkedIn && !checkedOut){
    if(!inside){if(!outsideSince)outsideSince=ms;if(ms-outsideSince>=5000){

checkedOut=true;checkOutAt=nowLocal();lastCheckoutMs=ms;overdueAlarm=false;confirmation="";addEvent("CHECKOUT");
    }}else outsideSince=0;
    if(!checkedOut && nowLocal()>=deadline){if(!overdueAlarm){overdueAlarm=true;addEvent("OVERDUE");}}
    else if(overdueAlarm){overdueAlarm=false;addEvent("ALARM_CLEARED");}
  }
  if(overdueAlarm)command="ALARM:ON";
  else if(confirmation.length() && ms-pendingConfirmationAt<10000)command=confirmation;
}
```

D.8 Event deduplication and spreadsheet escaping (storage.py)

The composite primary key and the formula escaping discussed in Section 2.7 and Section 3.16.

```
def safe_cell(v):
    if isinstance(v, str) and v.lstrip().startswith(('=', '+', '-', '@')):
        return "'" + v
    return v

    def event(self, value, received):
        data = dict(value)
        data['received_at'] = datetime.fromtimestamp(received, timezone.utc).isoformat()
        key = f"{data.get('source','receiver')}_{data['boot_id']}_{data['id']}"
```

```
        with self.connect() as db:
            return db.execute('INSERT OR IGNORE INTO events(key,received,data)
VALUES(?,?,?)',
                              (key, received, json.dumps(data, allow_nan=False))).rowcount > 0
```

D.9 Acquisition cycle with per-device failure isolation (app.py)

The polling cycle that keeps the last known values when a device stops answering, described in Section 3.16.

```
    def poll_once(self):
        now = time.time()
        if self.demo:
            self.demo_state(now)
            return
        futures = {role: self.pool.submit(self.fetch_device, role) for role in ('main',
'receiver')}
        for role, future in futures.items():
            try:
                data = future.result()
                with self.lock:
                    self.state[role] = data
                    self.state['links'][role] = {'online': True, 'received': now}
                    self.state['errors'].pop(role, None)
                if role == 'main':
                    if now-self.last_sample >= 10:
                        self.store.sample(data, now)
                        self.last_sample = now
                    seq = data.get('impact_sequence', 0)
                    if seq:
                        self.export_dirty |= self.store.event({'source': 'main', 'type': 'IMPACT',
                            'boot_id': data['boot_id'], 'id': seq,
                            'acceleration_g': data.get('impact_event_acceleration_g'),
                            'gyro_dps': data.get('impact_event_gyro_dps')}, now)
                else:
```

```
                for event in data.get('events', []):
                    self.export_dirty |= self.store.event(event, now)
                if not data.get('time_synced') and not self.clock_override:
                    self.command('time', {'epoch': int(now), 'tz': self.local_offset()})
            except Exception as exc:
                with self.lock:
                    self.state['links'][role] = {'online': False, 'received':
self.state['links'].get(role, {}).get('received')}
                    self.state['errors'][role] = str(exc)
        if self.export_dirty and now-self.last_export>10:
```

APPENDIX E: PHYSICAL TEST PROTOCOLS

The protocols below are written so that a later worker can obtain the outstanding measurements in Section 4.5 without redesigning the experiment, and each states its apparatus, its procedure and the quantity to be entered into the prepared table.

For the light response, the apparatus is a reference lux meter and a controllable room light. The helmet is placed with the BH1750 facing the same direction as the meter and at the same height. Readings are taken at descending illuminance from full room lighting to darkness, allowing five seconds at each step so that the one second acquisition interval has settled, and at each step the reported lux, the commanded PWM and the observed lamp condition are recorded. What is being checked is the mapping and the usability of the lamp, since the firmware clamps at 20 lux and 150 lux and applies an inverse quadratic between them.

For the gas response, the apparatus is a known source and adequate ventilation. Both MQ elements need their heaters powered for the manufacturer's preheat period before any reading is taken, which is the step most often skipped and which invalidates everything after it. Raw counts are logged in clean air to establish the baseline, then with the source present to establish the upper count, and the two are entered as the new constants.

For the impact chain the apparatus is a padded drop rig and a stopwatch, and the limits of the test must be stated in the record. This procedure verifies the detection logic and does not measure head injury. For the impact chain, record whether a resultant acceleration above 2.5 g creates a candidate event, whether the resultant gyroscope magnitude exceeds 250 deg/s during the approximately 300 ms confirmation window, whether the shared GPIO 33 buzzer remains active for 1.5 seconds, and whether additional impact detections are suppressed while the alert is active. After the alert ends, impact monitoring should resume. If the host sequence is also evaluated, report it as a separate integration check.

For the radio, the apparatus is a tape measure and a marked floor. With the helmet advertising, the filtered indication is recorded at 1 m and then at half metre steps out to 8 m, three times at each point. The measured power is taken from the 1 m readings and the exponent is fitted to the remainder. A walking test then crosses each

boundary slowly in both directions while the zone output is logged, to confirm that the hysteresis margin suppresses chatter.

For the battery, a fully charged pack is fitted and the system is left running with the lamp active and both heaters powered, with the reported percentage and the measured terminal voltage logged every fifteen minutes until the 6.40 V floor is reached.